

Treball de Fi de Grau

Grau en Enginyeria en Tecnologies Industrials (GETI)

Estudi i desenvolupament d'una arquitectura d'intel·ligència artificial neuromòrfica basada en xarxes neuronals de polsos (SNNs).

MEMÒRIA

22 d'abril de 2026

Autor: Alejandro Pérez Hernández

Director: Álvaro Gómez Pau

Convocatòria: data



ETSEIB

Escola Tècnica Superior
d'Enginyeria Industrial de Barcelona



Resum

1 Pàgina. Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetur at, consectetur sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

Índex

1	Prefaci	7
2	Introducció	9
2.1	Objectius	9
2.2	Abast del projecte	9
3	Context i marc teòric	11
3.1	Orígens de la Intel·ligència Artificial	11
3.2	Naixement xarxes neuronals	13
3.3	Machine Learning i Deep Learning	16
3.4	Xarxes neuronals artificials modernes (ANNs)	19
3.5	Limitacions de les ANNs	22
3.6	Xarxes neuronals de polsos (SNNs)	24
4	Fonament tècnics de les SNNs	25
4.1	Model de neurona de Hodgkin-Huxley	25
4.2	Model Integrate-and-Fire (IF) i Leaky Integrate-and-Fire (LIF)	26
4.3	Sinàpsis i transmissió de polsos en una SNN	29
4.4	Codificació i entrenament en xarxes neuronals de polsos	30
4.5	Entrenament de xarxes neuronals de polsos	32
5	Desenvolupament pràctic: Classificació amb ANN i SNN	35
5.1	Generació i transformació de les dades d'entrenament	35
5.2	Creació, entrenament i avaluació de la ANN	37
5.3	Conversió a SNN	39
5.4	Resultats i comparativa de rendiment	42
6	Desenvolupament pràctic: Aplicació del dataset MNIST amb ANN i SNN com a simulació de cas d'ús real	43
6.1	Justificació de l'ús de MNIST	43
6.2	Preparació de dades i entrenament de la ANN	43
6.3	Conversió a SNN i configuració temporal	43
6.4	Resultats i comparativa entre ANN i SNN	43
7	Resultats	43
8	Pressupost	45
9	Impacte ambiental	45
10	Conclusions	45
	Bibliografia	47
	Anex	51

Índex de figures

1	Una imatge del logo de l'ETSEIB	7
2	Model de neurona artificial de McCulloch i Pitts.	11
3	Il·lustració del funcionament de la Good Old-Fashioned AI (GOFAI).	12
4	Model de neurona del Perceptró.	14
5	Problemes linealment separables i no linealment separables. Font: [12].	14
6	Arquitectura d'un perceptró multicapa (MLP).	15
7	Flux de dades i actualització de pesos en la retropropagació.	16
8	Estructura bàsica d'una xarxa neuronal convolucional (CNN).	18
9	Estructura bàsica d'una xarxa neuronal recurrent (RNN).	19
10	Evolució del consum energètic en l'entrenament de models d'IA des de 2012 fins a 2020. Font: [25].	23
11	Circuit equivalent del model de Hodgkin-Huxley amb canals iònics dependents del voltatge. Font: [27]	26
12	Esquema elèctric conceptual d'una neurona Integrate-and-Fire (IF).	27
13	Evolució temporal del potencial de membrana $V(t)$ en una neurona IF i generació de polsos en superar el llindar V_{ref}	27
14	Esquema elèctric conceptual d'una neurona Leaky Integrate-and-Fire (LIF).	28
15	Evolució temporal del potencial de membrana $V(t)$ en una neurona LIF, mostrant l'efecte de la fuga i la generació de polsos. Font: [30]	28
16	Resposta temporal de la conductància sinàptica per a (a) una sinapsi exponencial i (b) una sinapsi alfa. Font: [31]	29
17	Comparació entre codificació per taxa i codificacions temporals. Font: [32]	32
18	Polígon generat per l'script de MATLAB.	36
19	Dades d'entrenament generades, amb punts de classe 0 (fora del polígon) en vermell i punts de classe 1 (dins del polígon) en vert.	36

1 Prefaci

Per no repetir informació és millor referir-se a altres apartats I recorda, sempre és important citar a la bibliografia [1].

La bibliografia ha d'estar ordenada, en teniu un exemple a la pàgina



Figura 1: Una imatge del logo de l'ETSEIB

2 Introducció

2.1 Objectius

2.2 Abast del projecte

Com hi ha coses escrites en aquest apartat, el següent començarà en una pàgina senar nova.

3 Context i marc teòric

Per entendre com funciona la intel·ligència artificial actual, i especialment aquelles basades en estructures que simulen el funcionament del cervell humà com les xarxes neuronals de polsos (SNNs), és essencial mirar enrere i comprendre d'on provenen, i com han estat concebudes des dels seus inicis. La manera en com es va definir i enfocar la intel·ligència artificial en els seus primers anys, així com la posterior evolució dels models d'intel·ligència computacional i els diferents mètodes d'aprenentatge emprats al llarg de la història, continuen tenint un profund impacte sobre les tecnologies i els paradigmes utilitzats avui dia. La història de la intel·ligència artificial no és només una successió d'avenços tècnics, sinó també un reflex dels diferents models amb què la ciència ha intentat descriure el pensament, l'aprenentatge, la intel·ligència i la consciència humana. Explorar els orígens de la IA ens permet identificar els seus punts forts, els seus límits, i explica per què, després de dècades de desenvolupament en una direcció, estem tornant a idees fortament inspirades en la neurobiologia. Aquesta retrospectiva ens ajuda a situar les arquitectures neuromòrfiques dins d'un context evolutiu, i a entendre com aquestes poden donar una resposta a les limitacions dels enfocaments tradicionals utilitzats en l'actualitat.

3.1 Orígens de la Intel·ligència Artificial

Els primers esbossos del que avui dia entenem com a intel·ligència artificial sorgeixen l'any 1943 quan, amb la premissa de crear un "cervell electrònic", el neurofisiòleg Warren McCulloch i el matemàtic Walter Pitts publiquen *A Logical Calculus of the Ideas Immanent in Nervous Activity* [2], un article que proposa un model matemàtic del sistema nerviós, a base d'una xarxa d'elements lògics simples, el que més tard es coneixerien com "Neurones Artificials". Aquestes neurones, tot i fer-ho d'una manera simple basada en la lògica matemàtica, funcionen de manera similar a com avui dia funcionen les xarxes neuronals tradicionals, agafant "l'input" o entrada, realitzant-li una suma ponderada en cada neurona amb una funció llindar que filtra el moviment d'informació entre neurones, i generant "l'output" o sortida desitjada, tal com es mostra a la [Figura 2](#).

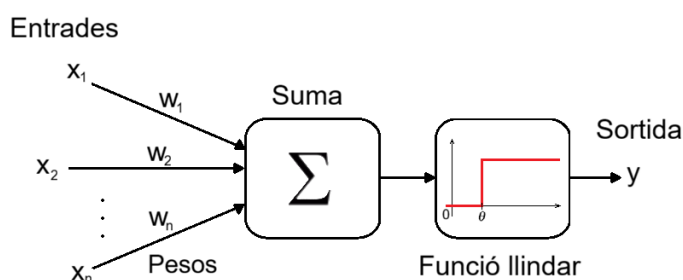


Figura 2: Model de neurona artificial de McCulloch i Pitts.

Així doncs, la sortida que produeixen n d'entrades quedaria definida per la següent funció:

$$f(\mathbf{x}) = h\left(\sum_{n=1}^N (x_n w_n) - \theta\right) \quad (1)$$

Aquest article marca l'inici i estableix les bases de les xarxes neuronals artificials, obrint la porta a la idea que el pensament pot ser simulat per una màquina.

El primer en plantejar aquest paradigma és Alan Turing en el seu article *Computing Machinery and Intelligence* [3]. En aquest, Turing planteja la pregunta de "Poden les màquines pensar?" introduint el que ara es coneix com el Test de Turing, un experiment de comportament que ell anomena *Imitation Game* (joc d'imitació). Amb aquest experiment proposa que, si una màquina pot imitar les respostes d'un humà de manera suficientment convincent perquè un humà no pugui distingir de manera fiable d'una persona, aquesta màquina podrà ser considerada intel·ligent. Turing també introdueix el concepte de *self-teaching machines*, proposant la idea d'una màquina capaç de ser entrenada de manera similar a un infant humà, a base de recompenses i/o càstigs.

Basant en les idees fonamentals introduïdes al article de Turing, la universitat de Dartmouth marca el naixement formal de la intel·ligència artificial com a camp al 1956 amb el *Dartmouth Summer Research Project on Artificial Intelligence* [4], un programa proposat per John McCarthy, Marvin Minsky, Nathaniel Rochester i Claude Shannon un any abans, que es basa en la conjectura de que tots els aspectes de l'aprenentatge humà es poden descriure amb prou precisió perquè una màquina el simuli. Aquest projecte reuneix als principals pensadors de les matemàtiques, la informàtica, la neurociència i la ciència cognitiva de la època, i estableix per a dècades de recerca i innovació en aquest nou camp.

Basant-se en els conceptes fonamentals introduïts per Turing i formalitzats durant el Dartmouth Summer Research Project on Artificial Intelligence, el camp de la intel·ligència artificial entra en la seva primera fase important: la IA simbòlica, també coneguda com a *Good Old-Fashioned AI* (GOFAI). Dominant la recerca en IA des de mitjans dels anys 50 fins a mitjans dels anys 90, aquest enfocament es centra en representar el coneixement mitjançant símbols llegibles pels humans i manipular-los mitjançant la lògica i la cerca heurística, tal i com s'il·lustra a la [Figura 3](#).

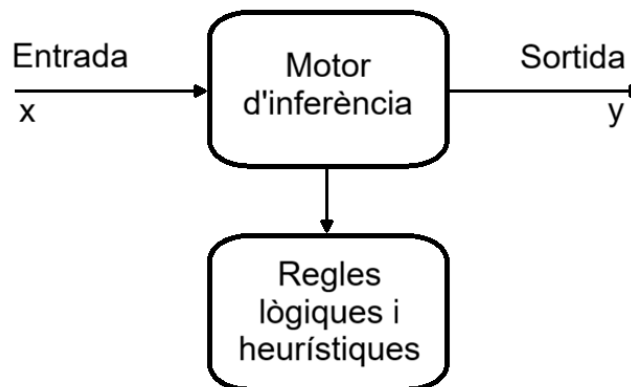


Figura 3: Il·lustració del funcionament de la Good Old-Fashioned AI (GOFAI).

Entre els primers programes importants basats en aquest enfocament s'inclou el *Logic Theorist* o *The Logic Theory Machine* [5] (1956), d'Allen Newell, Herbert A. Simons i Cliff Shaw, el primer programa en demostrar les teories proposades per Turing, aconseguint un sistema d'intel·ligència artificial capaç de raonament automatitzat demostrant teoremes matemàtics de *Principia Mathematica* [1]. Aprofitant l'èxit del programa, Newell, Simons i Shaw desenvolupen el *General Problem Solver* (GPS) [6], un sistema més versàtil que pretén resoldre una àmplia gamma de problemes mitjançant les estratègies heurístiques. Aquests esforços no només demostrin que les màquines poden realitzar tasques tradicionalment associades amb el raonament humà,

sinó que també inspiren futurs projectes de recerca i porten una creixent atenció per part del món acadèmic i del públic al camp emergent de la intel·ligència artificial.

Cimentant-se en els fonaments establerts per les primeres instàncies d'IA simbòlica i el treball teòric sobre neurones artificials realitzat per McCulloch i Pitts, la dècada dels 50 veu també l'aparició de màquines d'autoaprenentatge, programes amb la capacitat d'adaptar-se basats en experiències en lloc de basar-se únicament en regles fixes com les màquines tradicionals. El programa de dames d'Arthur Samuel (1952-1955) [7] marca un punt d'inflexió en el camp, demostrant l'habilitat de les màquines per aprendre i millorar sense ser explícitament programades per això, de manera similar al que proposava Turing d'ensenyar a una màquina com un infant. Més o menys al mateix temps, el model *Perceptron* [8] va ampliar les idees de les neurones McCulloch-Pitts a xarxes neuronals entrenables, destacant una via paral·lela al raonament simbòlic. Altres innovacions, com l'experiment de traducció automàtica de Georgetown-IBM (1954) [9], la primera demostració d'una màquina capaç de traduir frases senceres del rus a l'anglès, els primers programes de joc i el sistema *STUDENT* [10] per resoldre problemes d'àlgebra, van demostrar una creixent diversitat en els enfocaments de la IA, combinant mètodes estadístics, simbòlics i heurístics. Aquests desenvolupaments marquen el canvi del camp de la intel·ligència codificada a mà a sistemes capaços d'aprendre, adaptar-se i millorar, trets característics de la intel·ligència artificial moderna.

3.2 Naixement xarxes neuronals

Les xarxes neuronals són models computacionals inspirats pel funcionament del cervell humà. Consisteixen en grups de neurones artificials anomenades nodes, agrupades per capes. Aquestes, a través d'un *input* o entrada i mitjançant sumes ponderades de node a node, aconseguen processar informació. El primer model del que avui dia anomenariem xarxa neuronal apareix amb el model del Perceptró (*Perceptron* [8]) a mitjans dels anys 50. Concebut l'any 1957 per Frank Rosenblatt, el perceptró és un model de neurona artificial que utilitza un classificador binari conegut com a funció llindar (*threshold function*), una funció que utilitza un nombre d'entrades ($x_1 \dots x_n$), pesos ($w_1 \dots w_n$) i un valor llindar o biax (b), junt amb la funció de Heaviside, per determinar l'activació de la neurona. El seu funcionament és similar al de la neurona de McCulloch i Pitts [2], amb la diferència fonamental que, en aquest darrer model, els pesos associats a les entrades són fixos i definits manualment, mentre que en el perceptró aquests pesos són paràmetres entrenables que s'ajusten automàticament durant el procés d'aprenentatge.

$$f(\mathbf{x}) = h\left(\sum_{n=1}^N (x_n w_n) + b\right) \quad (2)$$

On $\sum_{n=1}^N (x_n w_n) + b = z$. D'aquesta manera, la neurona s'activa ($f(x) = 1$) si $z \geq 0$ i queda inactiva ($f(x) = 0$) si $z < 0$, tal i com s'il·lustra a la [Figura 4](#). Aquesta regla simple estableix si la informació s'ha de transmetre a la següent neurona o no, repetint aquest procés fins a arribar a la sortida (el valor de la darrera neurona) desitjada.

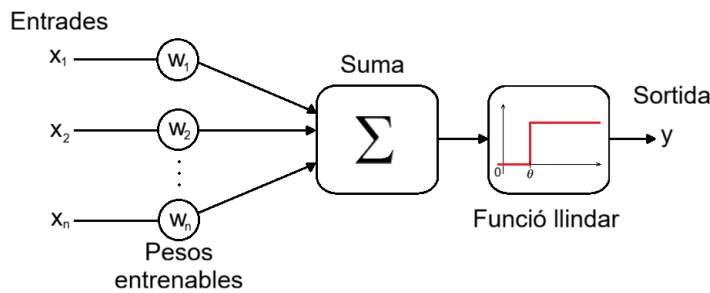


Figura 4: Model de neurona del Perceptró.

Aquest model de neurona possibilita la creació d'una xarxa interconnectada que permet per primera vegada que una màquina aprengués de manera autònoma un criteri de classificació a partir d'exemples, en lloc de dependre exclusivament de regles manuals escrites per humans. Concretament, possibilita ajustar pesos i biaix a partir de dades d'entrenament mitjançant una regla local d'actualització, i l'adaptació i millora amb experiència sense necessitat de reprogramació explícita. A més, opera de manera eficient i en línia, actualitzant pesos per cada exemple, cosa que el fa útil per a sistemes adaptatius en temps real. Al mateix any de la seva concepció, l'any 1957, Frank Rosenblatt simulà un model del Perceptró en un IBM 704 al laboratori aeronàutic de Cornell (*Calspan*), on es va provar la utilitat d'aquest model, el que serveix per cridar l'atenció del departament de la marina dels Estats Units (*ONR*), qui més tard finançaria el projecte per crear una màquina dissenyada específicament per executar aquests models, el *Mark I Perceptron*. Aquest model, però, no era pas perfecte. A causa de la seva arquitectura simple, el perceptró només pot separar les dades amb un hiperplà, és a dir, una línia (en 2D), un pla (en 3D), etc, com es mostra a la [Figura 5](#). Per tant, no podia resoldre problemes no linealment separables, cosa que va resultar problemàtica, al no poder resoldre la funció lògica bàsica XOR (exclusiva O). A més, la seva capacitat d'aprenentatge era limitada, doncs només podia aprendre patrons molt simples. Marvin Minsky i Seymour Papert van assenyalar totes aquestes debilitats al seu llibre *Perceptrons* (1969) [11], fet que va provocar que molts investigadors i finançadors perdessin l'interès en el camp de la intel·ligència artificial. Això va donar lloc a un període conegut com l'hivern de la IA, caracteritzat per la baixa inversió i la poca investigació i interès en aquest àmbit, el qual va allargar fins a mitjans dels anys 80.

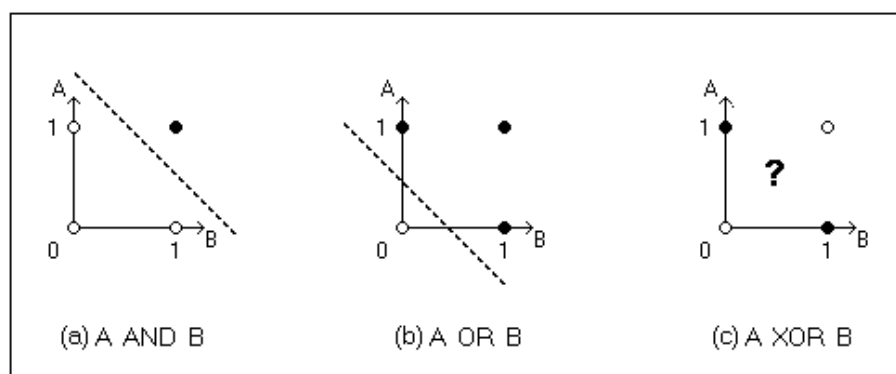


Figura 5: Problemes linealment separables i no linealment separables. Font: [12].

Després de la desil·lusió causada per les limitacions del perceptró simple, alguns investigadors

van continuar explorant la possibilitat de construir xarxes neuronals més potents. Ja a finals dels anys 50 i principis dels 60, es va començar a parlar de xarxes amb múltiples capes, però no es disposava d'un mètode eficient per entrenar-les, cosa que en limitava enormement l'aplicació pràctica. Tot i aquestes dificultats, la idea va quedar latent durant dècades fins que, l'any 1986, David E. Rumelhart, Geoffrey Hinton i Ronald J. Williams van publicar *Learning representations by back-propagating errors* [13], un article que redescobria i formalitzava l'algorisme de retropropagació de l'error (*backpropagation*) com a eina per entrenar xarxes neuronals multicapa. Aquest moment dona una base sòlida i funcional a un model que fins llavors havia estat només una idea prometedora: el perceptró multicapa (MLP).

El perceptró multicapa consisteix en una xarxa de neurones artificials organitzades en diverses capes consecutives. Aquest inclou una d'entrada, una o més capes ocultes, i una de sortida. Com està il·lustrat a la Figura 6, cada neurona d'una capa està connectada amb totes les neurones de la capa següent mitjançant pesos que, de manera similar al Perceptró simple, regulen la importància de cada connexió. Quan s'introdueixen dades a la xarxa, aquestes passen per les capes de manera seqüencial. Cada neurona calcula una combinació lineal de les seves entrades i hi aplica una funció d'activació no lineal, normalment la funció de Sigmoid [14] o la funció ReLU [15], que permet al sistema capturar patrons complexos.

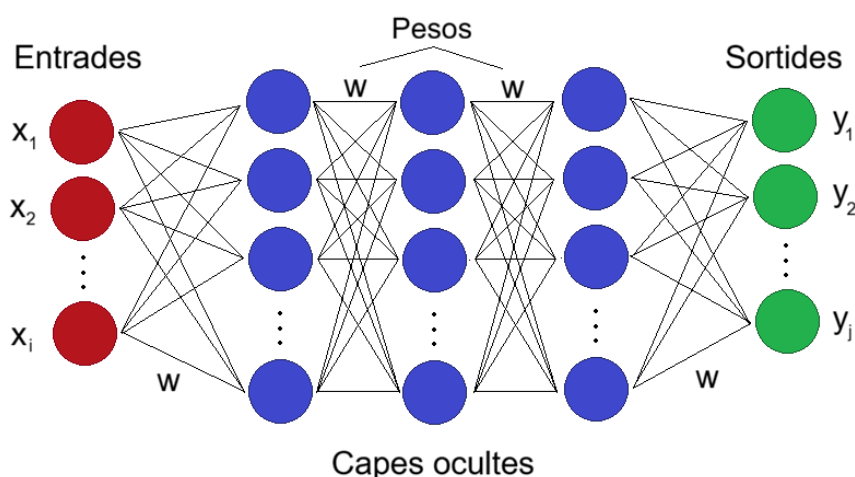


Figura 6: Arquitectura d'un perceptró multicapa (MLP).

El gran repte, però, era saber com ajustar els pesos per tal que la xarxa produís resultats correctes. La retropropagació va permetre solucionar aquest problema. Aquesta funciona de manera que, una vegada la xarxa produeix una sortida, es compara amb la sortida esperada i es calcula un error. Aquest error es propaga cap enrere des de la capa de sortida fins a les primeres capes, utilitzant el càlcul del gradient de l'error respecte a cada pes. Això permet modificar els pesos mitjançant un algorisme d'optimització com el descens del gradient, de manera que l'error es pugui anar reduint a cada iteració. Aquest procés es repeteix milers o milions de vegades, amb moltes dades diferents, fins que la xarxa aprèn a realitzar la tasca desitjada amb gran precisió. El flux de dades i l'actualització de pesos es pot veure esmentat a la Figura 7.

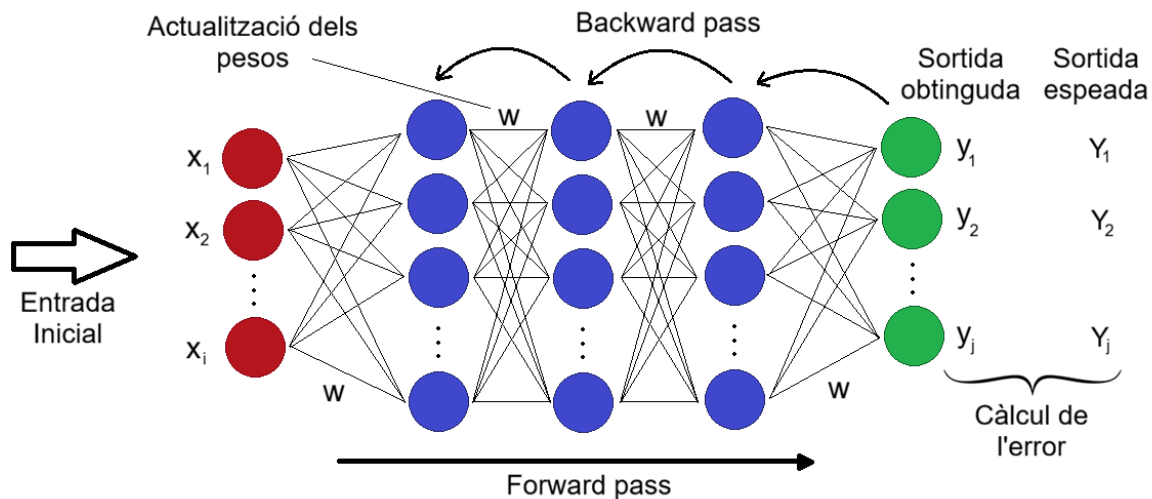


Figura 7: Flux de dades i actualització de pesos en la retropropagació.

Aquests avenços suposen una veritable revolució, doncs per primera vegada, les xarxes neuronals eren capaces de resoldre problemes complexos, no lineals, i amb moltes variables, cosa que va reactivar l'interès pel camp de la IA i va posar les bases del Deep Learning que avui coneixem.

3.3 Machine Learning i Deep Learning

El *Machine Learning* (o aprenentatge automàtic) és una branca de la intel·ligència artificial dedicada a l'estudi i al desenvolupament d'algorismes estadístics que aprenen a partir de dades. Aquestes algorismes permeten a les màquines generalitzar la informació obtinguda per resoldre tasques i prendre decisions en tasques per a les quals no han estat explícitament programades. El terme Machine Learning l'introdueix per primer cop Arthur Samuel a la dècada dels 50 per descriure programes o màquines capaces d'aprendre i millorar el seu rendiment mitjançant l'experiència. La primera demostració de que això era possible fou el ja mencionat programa de dames [7] del mateix Arthur Samuel. Durant les següents dècades, es van desenvolupar models i tècniques, com ara el perceptró i la retropropagació, que establirien els fonaments teòrics del Machine Learning, i popularitzarien el seu ús per l'entrenament de models computacionals durant la dècada dels 70. El Machine Learning utilitza principalment tres diferents paradigmes d'aprenentatge automàtic.

L'aprenentatge supervisat (SL), on l'algorisme aprèn a relacionar una entrada amb una sortida basant-se en conjunts d'exemples etiquetats, amb la finalitat de que aquest algorisme sigui capaç de predir etiquetes o valors per a noves dades de sortida en base a la entrada. Aquest model és el més estable, però no permet modelar decisions seqüencials.

L'aprenentatge no supervisat o aprenentatge autosupervisat (UL o SSL), que a diferència del supervisat, utilitza dades sense etiquetes per extraure estructures estadístiques o patrons de les dades. Aquests algorismes no tenen la mateixa capacitat per realitzar tasques concretes com els entrenats per aprenentatge supervisat i poden requerir d'un aprenentatge supervisat reduït per poder realitzar-les, però al no haver d'etiquetar les dades i relacionar-les manualment amb la sortida, són bastant més fàcils d'entrenar, el que redueix el cost econòmic i el temps de fer-ho,

especialment per a grans bases de dades no etiquetades.

Per últim, l'aprenentatge per reforç (RL), un mètode d'aprenentatge es basa en la psicologia conductista. En aquest, un agent o màquina rep informació de l'entorn (entrada), i emet accions (sortida). Un interprete, ja sigui humà o una funció de recompensa, retorna noves observacions (entrades), i una recompensa. L'objectiu de l'agent serà doncs aprendre un patró que maximitzi la recompensa acumulada, actualitzant de manera contínua els seus paràmetres segons l'experiència obtinguda. Aquest model, tot i ser ideal per resoldre problemes seqüencials i iteratius, és més costós, i pot presentar problemes d'inestabilitat i/o no convergència, els quals poden rendir el model inutilitzable per alguns usos. Cada paradigma té fonaments teòrics i aplicacions pròpies i ha evolucionat amb contribucions històriques que van definir les tècniques i els reptes actuals.

Malgrat l'estancament provocat per les limitacions del perceptró i la crítica de Minsky i Papert als anys 60, la introducció i formalització de mètodes d'entrenament per a xarxes multicapa (notablement la retropropagació a mitjans dels anys 80) i, més tard, la disponibilitat massiva de dades i capacitat de càlcul que apareixen amb la popularització de l'internet impulsen l'avenç i l'expansió del camp, donant lloc al que avui anomenem Deep Learning. El Deep Learning (DL) és una subàrea del Machine Learning que utilitza xarxes neuronals profundes, models multicapa amb moltes capes ocultes, normalment centenars o milers, per aprendre representacions jeràrquiques de les dades. La principal avantatge d'aquest tipus d'aprenentatge i de l'ús de xarxes neuronals profundes és la seva capacitat d'aprendre característiques directament a partir de dades brutes, com ara dades no etiquetades i no estructurades, el que les fa ideals per al seu entrenament amb algorismes d'aprenentatge no supervisat, aconseguint resultats similars als que es poden aconseguir utilitzant algorismes d'aprenentatge supervisat amb xarxes més simples, sense la necessitat d'estructurar i etiquetar les dades. Això es fa possible gràcies a les xarxes profundes desenvolupades als anys 80, com el perceptró multicapa, i a algorismes d'entrenament eficaços com la retropropagació combinada amb variants del descens de gradient (SGD, Adam, RMSprop) desenvolupats paral·lelament. Aquestes arquitectures permeten la creació d'aplicacions que modelaran les següents dècades de desenvolupament del camp d'estudi de la intel·ligència artificial. D'aquestes tecnologies, destaquen les CNNs, o xarxes neuronals convolucionals, les RNNs, o xarxes recurrents i variants, i més endavant els transformers.

Les CNNs, o xarxes neuronals convolucionals, són xarxes dissenyades especialment per processar dades amb estructura espacial o temporal, com ara imatges, vídeos o àudio, de manera eficient. Les CNNs, proposades per primer cop a l'any 1980 per Kunihiko Fukushima, amb el seu "neocognitron" [16], model que era basat en el seu model previ d'elements de llindar analògics per models de visió [17] presentat al 1969, i utilitzades per primer cop a l'any 1989 per Yann LeCun [18], amb el seu model LeNet-5, dissenyat per a la classificació de caràcters manuscrits, el primer exemple d'una xarxa convolucional aplicada amb èxit. Fukushima, basant-se en les observacions del neurocientífic David Hubel i el neurofisiòleg Torsten Wiesel sobre com certes neurones al cervell responen en exposar un individu a certs estímuls visuals específics, va proposar la idea que podria existir un model jeràrquic per processar la informació visual. El model que proposa utilitza capes de convolució, capes que apliquen filtres a l'entrada per extraure característiques locals. Funciona transformant un tensor d'entrada (per exemple, una imatge amb amplada x alçada x canals (com ara, els colors dels píxels)) en un conjunt de mapes d'activació (feature maps) mitjançant operacions lineals seguides d'una no-linearitat. Primer, aplica els filtres, que són petites matrius de pesos, per exemple de 3x3, a parts del tensor d'entrada, i calcula el producte. Aquest valor representa un punt del mapa d'activació. Aquest mateix

filtre es mou per calcular un altre punt de la entrada, fins a tenir tots els productes de tots els píxels de la imatge calculats, i amb tots aquests es forma el mapa. Aquest procés es repeteix per a cada un dels filtres, donant com a resultat diverses capes per cada neurona, cada una associada a un filtre, com està il·lustrat a la [Figura 8](#). Després, a aquestes capes se li aplica un biaix i una funció d'activació per introduir no linealitat a cada una. D'aquesta manera, podem aconseguir capes amb filtres més grans, que puguin detectar grans canvis de color en una imatge, permeten determinar contorns de grans objectes, i capes amb filtres més petits, que permeten diferenciar entre petits canvis a la imatge, cosa que, combinant-les, li dona l'habilitat a la xarxa d'identificació d'objectes en una imatge. És per això que aquests tipus de xarxes són les principalment utilitzades en el món de la visió per computador.

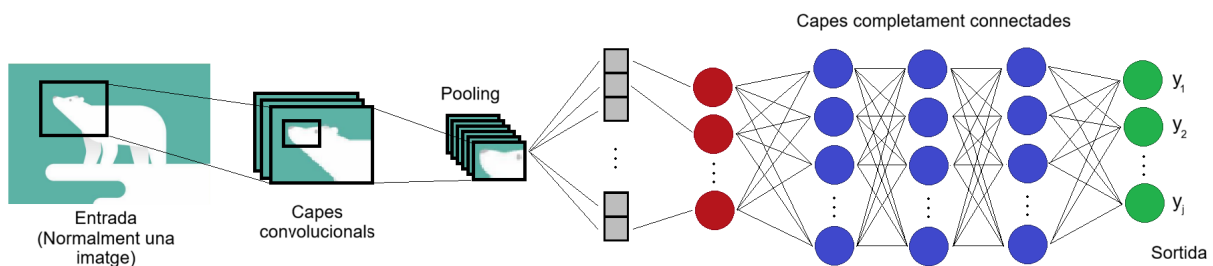


Figura 8: Estructura bàsica d'una xarxa neuronal convolucional (CNN).

Les RNNs, o xarxes neuronals recurrents, apareixen també a principis dels anys 1980, a mans de Rumelhart que, juntament amb David McClelland i Geoffrey Hinton, desenvolupa un algoritme de retropropagació a l'any 1986, presentat a l'article *Learning Representations by Back-Propagating Errors* [13]. A l'article, Rumelhart introdueix les xarxes neuronals recurrents com una extensió natural de les xarxes neuronals convencionals com l'MLP (Multi Layer Perceptrón), amb l'objectiu de poder tractar dades seqüencials o temporals, és a dir, informació on l'ordre dels elements és rellevant. A diferència de les xarxes tradicionals, on el flux d'informació va en una única direcció, d'una entrada cap a una sortida, les RNNs tenen connexions recurrents que permeten que la sortida d'una neurona en un període de temps pugui influir en el seu estat en períodes posteriors, és a dir, que la sortida de cada neurona no només depèn de la entrada, sinó que també depèn de les entrades ha tingut anteriorment, com es pot veure a la [Figura 9](#). D'aquesta manera, la xarxa manté una mena de "memòria" interna que li permet emmagatzemar informació del passat i utilitzar-la per processar noves entrades.

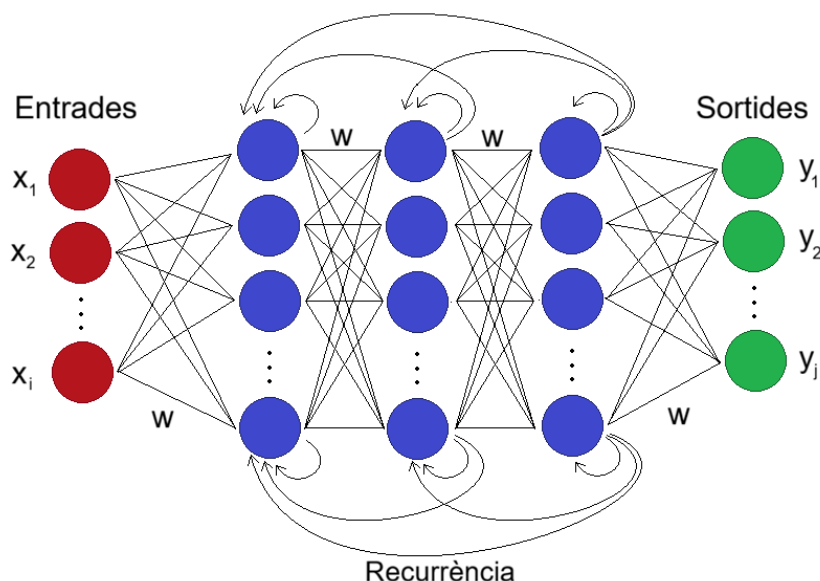


Figura 9: Estructura bàsica d'una xarxa neuronal recurrent (RNN).

Aquesta capacitat de recordar el context fa que les RNNs siguin ideals per realitzar tasques com el reconeixement de parla, la traducció automàtica o la predicció de sèries temporals, on les dependències temporals i de context són essencials. En comparació amb les xarxes neuronals estàtiques, les RNNs ofereixen una representació més dinàmica i flexible de les dades, podent modelar relacions al llarg termini dins d'una seqüència. Tot i això, malgrat el seu potencial, els primers models de RNN presentaven dificultats en l'entrenament, com el problema del gradient que s'esvaeix, on al entrenar una xarxa neuronal amb el mètode de retropropagació, a mesura que les dades es mouen d'una capa a una altra es van fer cada cop més petites, enlentint o directament parant el procés d'entrenament, o del gradient que explota, on les dades creixen de manera exponencial entre capes, produint el mateix efecte que el gradient que s'esvaeix, fets que limiten l'aplicació pràctica fins a la introducció de variants més estables com les LSTM i les GRU a finals dels anys 1990.

3.4 Xarxes neuronals artificials modernes (ANNs)

Les xarxes neuronals han recorregut un llarg camí des dels seus inicis, evolucionant en complexitat i eficiència amb l'avenç de la tecnologia i la investigació. Malgrat aquesta evolució, el fonament de les xarxes neuronals s'ha mantingut majorment inalterat. Les xarxes neuronals actuals es basen en els mateixos principis en els que es basaven les primeres xarxes neuronals, com ara el perceptró[8], i més tard els MLPs [13], les CNNs [18] i les RNNs. Aquestes xarxes estan estructurades en tres tipus de capes. La capa d'entrada rep les dades inicials, on cada neurona representa una característica específica dels "inputs". Les capes ocultes, que poden ser múltiples, són les responsables de processar aquestes dades i extreure característiques complexes a través de connexions ponderades. Cada neurona d'una capa oculta combina les entrades que rep, aplica un pes i una funció d'activació abans d'enviar la informació a la següent capa. Finalment, la capa de sortida presenta el resultat del model, que pot ser una classificació, una predicció numèrica o qualsevol altra forma de resultat.

Els pesos assignats a cada connexió determinen la influència de l'entrada sobre la sortida. A

més, les neurones poden disposar de biaixos, que afegeixen una constant per assegurar que les sortides puguin ser ajustades independentment de les entrades. Les funcions d'activació són fonamentals per a la creació de no linearitats dins del model, permetent que les xarxes aprenguin relacions complexes. Algunes de les funcions d'activació més populars avui dia inclouen la ReLU (Rectified Linear Unit), que retorna 0 per a entrades negatives i la pròpia entrada per a entrades positives, la sigmoide, que comprimeix les sortides entre 0 i 1, i la tanh, que les comprimeix entre -1 i 1.

El procés d'aprenentatge d'una ANN moderna es realitza en diversos passos. Primer, es fa una propagació cap endavant, on les dades es transmeten de la capa d'entrada fins a la sortida, passant per les capes ocultes i aplicant les funcions d'activació. Després, es calcula l'error comparant la sortida obtinguda amb la sortida esperada. Finalment, es realitza la retropropagació, un procés mitjançant el qual els pesos i biaixos són ajustats per reduir l'error. Utilitzant tècniques com el gradient descendent, la xarxa actualitza els pesos de manera que les futures prediccions siguin més precises.

Des dels anys 90, les xarxes neuronals han experimentat una evolució significativa no tant per la tecnologia en si, sinó pel seu escalat en termes de complexitat i capacitat. Inicialment, aquestes xarxes podien tenir centenars de milers de paràmetres (o neurones), limitades per la disponibilitat de dades i la potència de càlcul de la època. Amb l'accés a grans quantitats de dades que sorgeix amb l'aparició i popularització d'internet, i a maquinaria més potent, com les unitats de processament gràfic (GPUs), les xarxes neuronals han pogut créixer fins a arribar als milers de milions de paràmetres, permetent un modelatge més precís i detallat de patrons complexos, com ara la parla. Aquest increment en l'escala ha facilitat el desenvolupament d'intel·ligència artificial més avançada, capaç d'aprendre de conjunts de dades massius i de realitzar tasques que abans eren impensables, com ara el reconeixement d'imatges a gran escala, la traducció automàtica precisa i la generació de contingut de forma creativa.

Les arquitectures convencionals com les CNNs i les RNNs continuen utilitzant-se per les tasques pels quals van ser dissenyades (CNNs per visió, RNNs per dades seqüencials), però al mateix temps han aparegut alternatives sovint més eficients o millor adaptades a problemes moderns, com els transformers per NLP (processament de llenguatge natural) i seqüències llargues, variants d'object-detection com YOLOv8 [19] per detecció d'objectes en temps real en visió per ordinador, GANs (Generative Adversarial Networks) [20] per generació de contingut sintètic com imatges o vídeos i arquitectures híbrides que combinen components (per exemple: CNN+transformer), que en molts casos superen o complementen les CNN/RNN tradicionals, i que poc a poc van reemplaçant a aquestes xarxes. En els últims anys han estat sobretot els transformers els que s'han imposat com l'arquitectura més utilitzada i sovint la més efectiva en moltes tasques (especialment en NLP, però també en visió i multimodalitat), en part per la popularitat que intel·ligències artificials de processament de llenguatge basades en aquesta arquitectura han guanyat en aquest període, com ara el ChatGPT o Claude.

Els transformers apareixen fa relativament poc, més concretament a l'any 2017, amb la publicació de l'article *Attention Is All You Need* de Vaswani et al. [21]. En aquest article, s'introdueixen els transformers com una nova arquitectura de xarxa neuronal basada exclusivament en mecanismes d'atenció, eliminant així la necessitat de recurrència o convolució, que fins al moment caracteritzen les arquitectures anteriors com les RNNs o CNNs.

Els transformers es basen en el mecanisme d'atenció, en concret en el mecanisme d'autoatenció, el qual permet a cada element d'una seqüència ponderar la rellevància de tots els altres elements

de la mateixa seqüència per generar la seva pròpia representació.

Aquest procés es duu a terme mitjançant capes d'atenció múltiple (multi-head attention layers), que permeten que el model aprengui diferents tipus de relacions i dependències entre paraules o unitats de dades de manera simultània. A diferència de les RNNs, que processen les dades seqüencialment, els transformers poden processar tots els elements en paral·lel, fet que redueix significativament el temps d'entrenament i millora l'escalabilitat del model.

A més del mecanisme d'atenció, els transformers incorporen una sèrie de components addicionals com les "positional encodings" o codificacions posicionals, que afegeixen informació sobre la posició relativa dels elements dins la seqüència, i les "feed-forward networks" o xarxes de prealimentació, capes completament connectades que operen sobre cada posició de manera independent. Aquesta combinació de components fa que els transformers siguin extremadament flexibles i adaptables a diferents tipus de dades i tasques.

Els avantatges dels transformers són nombrosos. Ofereixen un rendiment superior en tasques de processament de llenguatge natural (NLP), permeten entrenar models amb una quantitat massiva de dades i paràmetres, i són altament paral·lelitzables, cosa que redueix els costos computacionals en comparació amb altres arquitectures seqüencials. Aquestes característiques han impulsat el desenvolupament dels grans models de llenguatge (LLMs), com la sèrie GPT (Generative Pre-trained Transformer), BERT (Bidirectional Encoder Representations from Transformers) o T5 (Text-To-Text Transfer Transformer), que han revolucionat el camp de la intel·ligència artificial moderna.

Avui dia, els transformers constitueixen la base de la majoria dels avenços en IA, no només en processament de llenguatge, sinó també en visió per computador, bioinformàtica, generació de codi i àrees multimodals que combinen text, imatge, àudio i vídeo. La seva capacitat per aprendre representacions contextuais complexes i generalitzar coneixement a tasques diverses els ha convertit en una de les fites més significatives en la història de la intel·ligència artificial i el deep learning, marcant l'inici d'una nova era dominada pels models fundacionals i els sistemes d'aprenentatge a gran escala.

Una part substancial del progrés recent en xarxes neuronals, incloent-hi l'aparició i consolidació dels transformers, no pot atribuir-se únicament a innovacions en l'arquitectura dels models. Aquesta evolució ha estat possible gràcies a un cúmul de factors, entre els quals destaca la millora significativa de les eines de programació, la disponibilitat d'infraestructures computacionals d'alt rendiment i l'accés a volums massius de dades digitalitzades. En aquest context, els llenguatges de programació actuals han tingut un paper fonamental. Python, en particular, s'ha posicionat com el llenguatge predominant en "machine learning" no només per la seva sintaxi accessible, sinó també per la maduresa d'un ampli conjunt de biblioteques especialitzades que faciliten totes les etapes del cicle de vida dels d'aquests models. Eines com TensorFlow, Keras, Pytorch i JAX proporcionen eines d'alt nivell que permeten definir i treballar amb arquitectures complexes d'una manera molt més senzilla i estructurada, optimitzar sèries numèriques de manera eficient i la integració nativa amb GPU i TPU, fet que permet entrenar models de gran escala que haurien estat inviables fa només uns anys. Paral·lelament, l'existència de paquets orientats a la gestió i preprocesament de dades com NumPy, Pandas o Dask, ha fet possible organitzar, transformar i manipular volums ingents d'informació d'una manera sistemàtica i reproducible, cosa que ha facilitat notablement l'entrenament d'aquests models. Tot plegat ha generat un entorn on el desenvolupament, experimentació i validació de models complexos és

molt més accessible, accelerant la innovació i contribuint directament a l'expansió de mètodes d'aprenentatge profund cada vegada més sofisticats.

Amb l'aparició de noves arquitectures neuronals i tecnologies d'aprenentatge automàtic, també han sorgit mètodes més especialitzats per classificar i avaluar els models segons les necessitats i la naturalesa de cada tasca. En el cas de la classificació, avui dia s'utilitzen mètriques com la precisió de la resposta obtinguda, el recall, que mesura la capacitat del model per identificar correctament els elements positius d'una classe, i la F1-score, que representa la mitjana harmònica entre precisió i recall, equilibrant els dos aspectes quan cap d'ells no pot ser ignorat, que permeten analitzar diversos aspectes del comportament del model, especialment en situacions amb classes desequilibrades. Per a tasques de regressió, en canvi, és habitual emprar mesures com l'error quadràtic mitjà (MSE) o l'error absolut mitjà (MAE), que quantifiquen la magnitud de les discrepàncies entre les prediccions i els valors reals. També s'utilitzen eines més específiques, com la matriu de confusió en problemes de múltiples classes, l'AUC-ROC (Area Under the Receiver Operating Characteristic Curve) [22] en classificació binària, el BLEU (Bilingual Evaluation Understudy) [23] en traducció automàtica o el mAP (mean Average Precision) [24] en detecció d'objectes, cadascuna pensada per capturar aspectes concrets del rendiment. L'elecció d'aquestes mètriques és essencial, ja que proporciona una visió precisa sobre la qualitat del model i orienta les millores necessàries per adequar-lo als objectius de l'aplicació concreta.

Un exemple clàssic que il·lustra de manera integrada els principis exposats fins ara és el conjunt de dades MNIST, un conjunt format per imatges de dígitos manuscrits. Aquest recurs, àmpliament utilitzat en entorns educatius i experimentals, permet observar com diferents arquitectures, des de perceptrons multicapa fins a xarxes convolucionals, aborden una mateixa tasca de classificació. Per exemple, un model inicialment simple pot constar d'una capa d'entrada de 784 neurones (derivades dels 28x28 píxels que té cada imatge), una o diverses capes ocultes i una capa de sortida amb 10 neurones corresponents a les classes possibles (números del 0 al 9). Mitjançant el procés d'entrenament basat en propagació cap endavant i retropropagació del gradient, la xarxa aprèn a extreure patrons visuals rellevants a partir de milers d'imatges etiquetades. En aquest treball aquest mateix conjunt s'utilitzarà de manera il·lustrativa per demostrar com funcionen exactament les xarxes prèviament tractades, i per exemplificar les avantatges, desavantatges i diferències que poden tenir les xarxes de polsos respecte a les xarxes neuronals convencionals ja comentades.

3.5 Limitacions de les ANNs

Les xarxes neuronals artificials, malgrat els grans avenços i l'eficàcia demostrada en nombroses aplicacions, presenten limitacions que condicionen el seu desplegament i la seva adaptabilitat a certs entorns. En primer lloc, el cost computacional i energètic associat a l'entrenament de models grans és molt elevat. L'entrenament de xarxes amb milions o milers de milions de paràmetres requereix centres de càlcul amb GPUs/TPUs especialitzades, hardware que requereix d'un gran consum d'electricitat, i un temps significatiu d'entrenament, cosa que suposa costos econòmics i ambientals notables. A la [Figura 10](#) es pot veure com ha evolucionat aquest consum a mesura que han evolucionat els models per a realitzar progressivament tasques de més complexitat. Això també es tradueix en una barrera d'entrada per a organitzacions petites i investigadors amb recursos limitats.

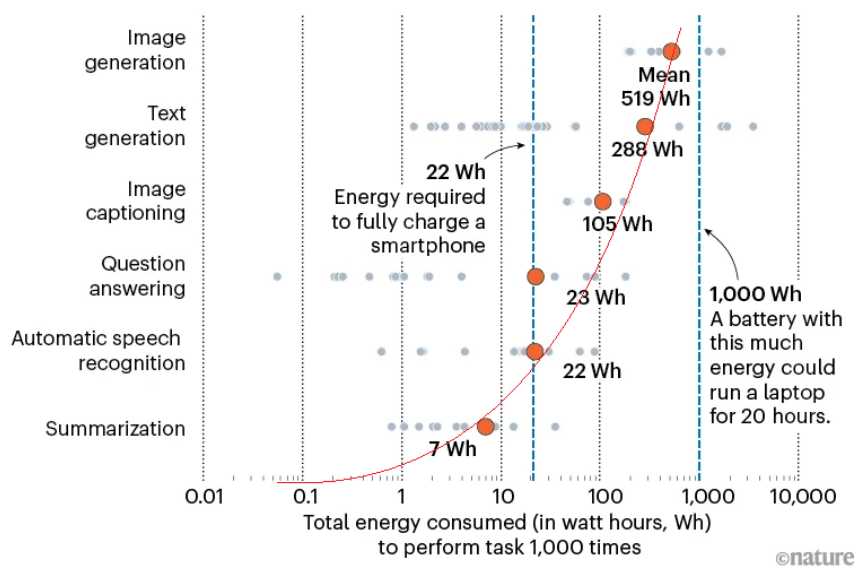


Figura 10: Evolució del consum energètic en l'entrenament de models d'IA des de 2012 fins a 2020. Font: [25].

Una altra limitació és la manca de temporalitat natural en moltes arquitectures estàndard. Tot i que les RNNs i variants com LSTM/GRU van ser concebudes per tractar dependències temporals, moltes arquitectures modernes, com per exemple els transformers, requereixen de mecanismes addicionals per captar de forma explícita la dinàmica temporal en dades contínues o en fluxos en temps real, fet que pot complicar el disseny de models per aplicacions com control en temps real o sèries temporals amb dependències llargues i no estacionàries.

Pel que fa a la correspondència amb el cervell humà, les ANNs no reproduïxen la major part dels principis biològics del processament neuronal. El seu funcionament és una aproximació matemàtica inspirada en el cervell, però diferent en arquitectura. Això implica que, malgrat algunes capacitats anàlogues (com l'aprenentatge a partir d'exemples), les xarxes artificials no presenten les mateixes propietats de robustesa, plasticitat i eficiència que caracteritzen el cervell humà.

L'escalabilitat amb eficiència és un repte addicional. L'augment de grandària dels models (per exemple, en LLMs) acostuma a portar millors resultats, però la millora no és lineal respecte al nombre de paràmetres. Sovint cal afegir ordres de magnitud més de paràmetres per obtenir guanys incrementals de precisió. En models molt grans com els LLMs actuals, per millorar notablement la qualitat cal incorporar molts més paràmetres, fet que fa que el cost d'entrenament i el consum energètic augmentin de manera quasi-exponencial respecte a les millores obtingudes. Això empitjora la relació cost-benefici i complica la gestió pràctica, fent necessari revisar dissenys, tècniques d'eficiència (poda, quantització, knowledge distillation) i estratègies d'entrenament per contenir costos i consum, i limita l'escalament d'aquests models passat de models massius.

Finalment, hi ha limitacions rellevants per a "edge computing" o computació perifèrica, on el processament de dades i la presa de decisions es fan pròxims a la font de les dades per reduir la latència, i sistemes embarcats. La majoria de models d'alt rendiment són massa grans o exigents per a dispositius amb recursos limitats (CPU de baixa potència, memòria reduïda, restriccions

d'energia), cosa que obliga a recórrer a tècniques de compressió, quantització, poda o a dissenys específics "light-weight" o de baix pes o cost computacional, que sovint impliquen una pèrdua de precisió o capacitat. Aquestes restriccions condicionen l'adopció de les ANNs en aplicacions mòbils, IoT (Internet of Things) i en entorns amb requisits estrictes d'energia i latència.

3.6 Xarxes neuronals de polsos (SNNs)

Les SNNs, o Xarxes neuronals de polsos, apareixen com una evolució de les xarxes neuronals artificials tradicionals amb l'objectiu d'apropar encara més el funcionament dels models computacionals al comportament real del cervell humà. La seva motivació principal és neuromòrfica, és a dir, s'inspiren directament en la manera com les neurones biològiques processen i transmeten la informació, mitjançant impulsos elèctrics discrets, coneguts com a spikes o polsos. A diferència de les ANNs convencionals, que operen amb valors continus i activacions instantànies, les SNNs introdueixen explícitament la dimensió temporal com a part fonamental del càlcul neuronal.

En aquest tipus de xarxes, la informació no es codifica únicament en la magnitud del senyal, sinó també en el temps en què es produeixen els polsos. Aquesta codificació temporal permet representar la informació d'una manera molt més eficient i expressiva, ja que l'instant d'emissió d'un impuls pot contenir tanta o més informació que el seu valor. Les neurones acumulen el potencial d'entrada al llarg del temps i només emeten un pols quan aquest supera un cert llindar, imitant així el comportament de les neurones biològiques. D'aquesta manera, el processament esdevé inherentment dinàmic i dependent del temps, fet que fa que les SNNs siguin especialment adequades per tractar senyals temporals continus, com ara dades sensorials, fluxos d'esdeveniments o informació procedent de sensors neuromòrfics.

En comparació amb les ANNs tradicionals, les SNNs presenten diversos avantatges destacables. En primer lloc, permeten un consum energètic molt inferior, ja que només es produeix activitat quan hi ha polsos, fet que les fa especialment atractives per a sistemes encastats de baix consum, i per models de gran tamany, on la utilització de ANNs convencionals significa un gran cost energètic. En segon lloc, ofereixen una representació temporal més rica de la informació, cosa que facilita la captura de patrons temporals complexos sense necessitat de mecanismes addicionals. A més, la seva naturalesa basada en esdeveniments permet un processament més eficient i proper al funcionament del cervell, tant en termes de paral·lelisme com d'escalabilitat.

Tot i aquests avantatges, les SNNs també presenten desafiaments importants. L'entrenament d'aquestes xarxes és més complex que en les ANNs convencionals, ja que els mètodes tradicionals com la retropropagació no es poden aplicar directament a causa de la naturalesa discreta dels polsos. A més, la implementació pràctica de SNNs sovint requereix hardware especialitzat, com ara xips neuromòrfics, per aprofitar al màxim els seus avantatges energètics i de processament. Malgrat aquests reptes, les SNNs representen una àrea de recerca molt activa i prometedora dins del camp de la intel·ligència artificial i el processament neuromòrfic, amb aplicacions potencials en robòtica, sistemes embarcats, i interfícies cervell-màquina, entre altres.

4 Fonament tècnics de les SNNs

Els primers models de neurona que avui dia s'utilitzen per la modelització de xarxes neuronals de polsos sorgeixen a l'any 1907 a mans de Louis Lapicque i, posteriorment, l'any 1952 a mans d'Alan Hodgkin i Andrew Huxley com a models matemàtics i electrofisiològics per a la simulació del funcionament d'una neurona humana, no amb la intenció de crear una intel·ligència artificial basada en aquestes. Aquests models neixen en el context de la neurociència amb l'objectiu d'entendre com les neurones biològiques processen i transmeten informació mitjançant senyals elèctrics. No és fins dècades més tard que aquests models són recuperats i reinterpretats com a fonament de les xarxes neuronals de polsos (SNNs).

Dins dels múltiples models proposats al llarg del segle XX, dos han esdevingut especialment rellevants per al desenvolupament de les SNNs, el model de Hodgkin-Huxley, d'alta fidelitat biològica, i el model Integrate-and-Fire, en particular la seva variant *Leaky Integrate-and-Fire* (LIF), àmpliament utilitzada en models computacionals per la seva simplicitat i eficiència.

4.1 Model de neurona de Hodgkin-Huxley

El model de Hodgkin-Huxley, publicat l'any 1952 [26], és el primer model quantitatiu capaç de descriure amb precisió el comportament elèctric d'una neurona biològica. Aquest model va ser desenvolupat a partir d'experiments amb l'axó d'un calamar i descriu la generació i propagació del potencial d'acció mitjançant corrents iònics dependents del voltatge a través de la membrana neuronal.

A nivell elèctric, la membrana neuronal es modela com un circuit equivalent format per una capacitat (la membrana lipídica) en paral·lel amb diverses conductàncies variables que representen canals iònics de sodi (Na^+), potassi (K^+) i un corrent de fuga (leak). La dinàmica del potencial de membrana $V(t)$ ve donada per la següent equació diferencial:

$$C_m \frac{dV}{dt} = I(t) - g_{Na} m^3 h (V - E_{Na}) - g_K n^4 (V - E_K) - g_L (V - E_L) \quad (3)$$

on C_m és la capacitat de la membrana, $I(t)$ el corrent injectat, g_{Na} , g_K i g_L són les conductàncies màximes dels canals de sodi, potassi i fuga respectivament, i E_{Na} , E_K i E_L els seus potencials d'equilibri. Les variables m , h i n són variables dinàmiques que representen l'estat d'obertura dels canals iònics i segueixen equacions diferencials pròpies dependents del voltatge. El circuit equivalent es mostra a la [Figura 11](#).

Aquest model reproduïx amb gran fidelitat fenòmens biològics com el llindar d'activació, el període refractari i la forma exacta del potencial d'acció. Tot i això, el seu elevat cost computacional el fa poc adequat per a xarxes neuronals grans, motiu pel qual rarament s'utilitza directament en SNNs a gran escala.

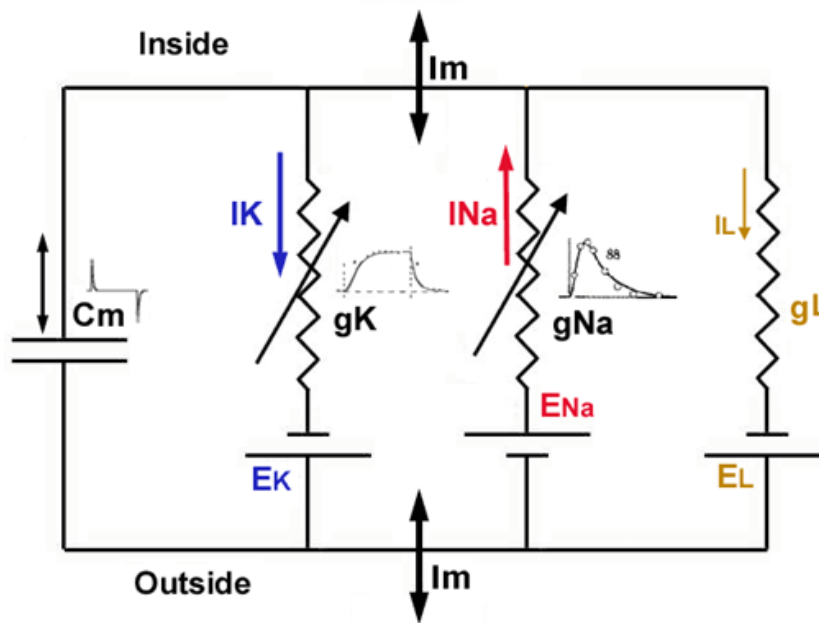


Figura 11: Circuit equivalent del model de Hodgkin-Huxley amb canals iònics dependents del voltatge. Font: [27]

4.2 Model Integrate-and-Fire (IF) i Leaky Integrate-and-Fire (LIF)

El model Integrate-and-Fire va ser introduït originalment per Louis Lapicque l'any 1907 al seu article *Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation* [28], i posteriorment refinat per Stein en el seu article *A theoretical analysis of neuronal variability* [29] en la seva variant més utilitzada, el model *Leaky Integrate-and-Fire*. Aquest model de neurona és una simplificació extrema del comportament neuronal, però captura els mecanismes essencials necessaris per al càlcul en SNNs.

A nivell elèctric, la neurona Integrate-and-Fire (IF) es pot modelar com un sistema que integra el corrent d'entrada al llarg del temps i genera un pols discret quan el potencial acumulat supera un determinat llindar. El corrent d'entrada $I(t)$ pot provenir d'una font externa, en el cas d'una neurona d'entrada de la xarxa, o bé correspondre a la suma ponderada dels polsos generats per les neurones de la capa anterior. Addicionalment, el model pot incorporar un terme de biaix, representable com una font de corrent constant I_b , que permet desplaçar el punt de funcionament de la neurona.

Aquest corrent s'integra mitjançant un condensador, de manera que la tensió associada al potencial de membrana augmenta proporcionalment a la càrrega acumulada. El valor de tensió resultant es compara amb una referència mitjançant un amplificador operacional actuant com a comparador, que defineix el llindar d'actuació V_{ref} . Quan la tensió supera aquesta referència, el comparador genera el pols de sortida i s'aplica un mecanisme de reinici del potencial. El esquema elèctric conceptual d'una neurona IF es mostra a la Figura 12.

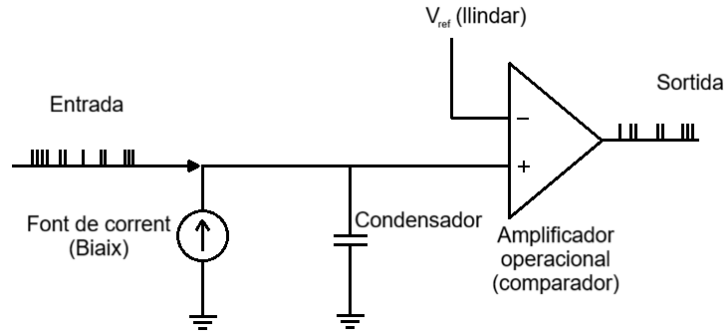


Figura 12: Esquema elèctric conceptual d'una neurona Integrate-and-Fire (IF).

Des d'un punt de vista matemàtic, la dinàmica del potencial de membrana $V(t)$ en el model IF ve descrita per:

$$C \frac{dV}{dt} = I(t) + I_b \quad (4)$$

on C és la capacitat equivalent, $I(t)$ representa el corrent d'entrada i I_b el terme de biaix, si s'utilitza. En absència d'entrada neta, el potencial es manté constant, ja que el model no incorpora cap mecanisme de descàrrega.

Quan el potencial de membrana assoleix el llindar V_{ref} , la neurona emet un pols i el potencial es reinicia a un valor V_{reset} :

$$\text{si } V(t) \geq V_{ref} \Rightarrow \begin{cases} \text{emet pols} \\ V(t) \leftarrow V_{reset} \end{cases}$$

A la [Figura 13](#) es pot observar una representació temporal del potencial de membrana i dels polsos generats per una neurona IF en resposta a un corrent d'entrada.

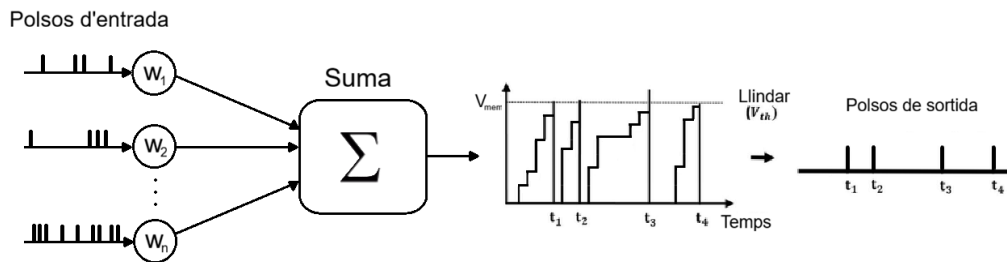


Figura 13: Evolució temporal del potencial de membrana $V(t)$ en una neurona IF i generació de polsos en superar el llindar V_{ref} .

Aquest mecanisme introdueix una no-linealitat temporal i converteix la sortida de la neurona en una seqüència discreta d'esdeveniments temporals. No obstant això, el model IF presenta

una limitació rellevant. Qualsevol corrent positiu sostingut, encara que sigui molt petit, acaba provocant un dispar, ja que la integració és indefinida en el temps.

Per corregir aquest comportament, Stein introdueix la neurona Leaky Integrate-and-Fire (LIF), que amplia el model IF afegint un mecanisme de pèrdua del potencial de membrana. A nivell elèctric, aquesta extensió es correspon amb la incorporació d'una resistència en paral·lel amb el condensador, que modela les pèrdues naturals de la membrana i força el potencial a relaxar-se cap a un valor de repòs V_{rest} . El esquema elèctric conceptual d'una neurona LIF es mostra a la [Figura 14](#).

Figura 14: Esquema elèctric conceptual d'una neurona Leaky Integrate-and-Fire (LIF).

La dinàmica del potencial de membrana en el model LIF es descriu mitjançant:

$$\tau_m \frac{dV}{dt} = -(V - V_{reset}) + R(I(t) + I_b) \quad (5)$$

on $\tau_m = RC$ és la constant de temps de la membrana i R la resistència equivalent. A diferència del model IF, en absència d'entrada el potencial decau exponencialment cap a V_{reset} , evitant la integració il·limitada.

De manera anàloga al cas IF, una representació temporal del potencial de membrana i dels polsos associats, com la mostrada a la [Figura 15](#), resulta especialment útil per evidenciar l'efecte de la fuga i la relaxació del potencial entre entrades consecutives.

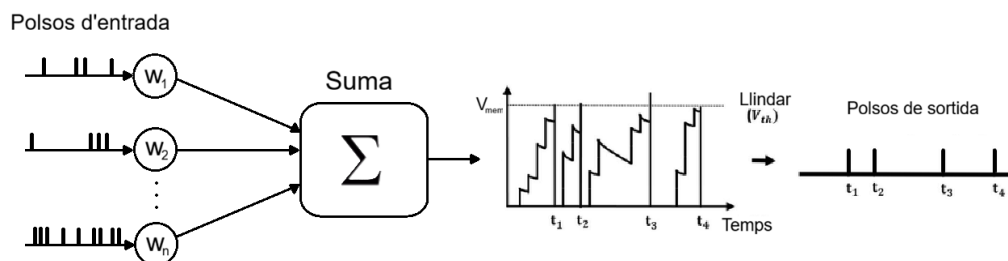


Figura 15: Evolució temporal del potencial de membrana $V(t)$ en una neurona LIF, mostrant l'efecte de la fuga i la generació de polsos. Font: [\[30\]](#)

Aquesta extensió aporta una millor estabilitat temporal, una major robustesa davant soroll i un comportament més proper al de les neurones biològiques, tot mantenint una elevada eficiència computacional. Per aquest motiu, el model LIF s'utilitza de manera majoritària en la simulació de xarxes neuronals espigades i en implementacions sobre hardware neuromòrfic.

4.3 Sinàpsis i transmissió de polsos en una SNN

Tot i que els models de neurona IF i LIF defineixen la dinàmica del potencial de membrana i el mecanisme de dispar, el comportament d'una xarxa neuronal de polsos ve determinat principalment pel model sinàptic. Les sinapsis són les encarregades de convertir els polsos presinàptics en corrents d'entrada que modulen el potencial de les neurones, actuant com l'element intermedi que connecta l'activitat discreta de la xarxa amb la dinàmica contínua del potencial, es a dir, l'encarregat de traduir la informació dels polsos a les neurones.

En una SNN, cada pols presinàptic no provoca un increment del potencial de membrana, sinó que genera un corrent transitori. Aquest corrent pot modelar-se mitjançant una funció de resposta temporal, habitualment anomenada kernel sinàptic. D'aquesta manera, el corrent sinàptic total que rep una neurona en un instant donat pot expressar-se com:

$$I_{syn}(t) = \sum_{i,j} w_{ij} \sum_k \kappa(t - t_j^k) \quad (6)$$

on w_{ij} és el pes de la sinapsi que connecta la neurona presinàptica j amb la neurona postinàptica i , t_{pols}^k representa els instants de dispar dels polsos presinàptics, i κ és el kernel sinàptic que descriu la forma temporal del corrent generat per un sol pols. Aquest kernel pot adoptar diferents formes, sent les més comunes la funció exponencial, definida com:

$$\kappa_{exp}(t) = \frac{1}{\tau_s} e^{-\frac{t}{\tau_s}} u(t) \quad (7)$$

On τ_s és la constant de temps sinàptica i $u(t)$ és la funció d'esglaó unitari o funció de Heaviside, que garanteix la causalitat del model, i la funció alfa, definida per:

$$\kappa_{alpha}(t) = \frac{t}{\tau_s} e^{-\frac{t}{\tau_s}} u(t) \quad (8)$$

La funció exponencial modela un corrent que augmenta bruscament en el moment del pols i decau exponencialment després, mentre que la funció alfa presenta un augment més gradual seguit d'una decaiguda, oferint una representació més realista de la resposta sinàptica biològica. A la [Figura 16](#) es poden observar les formes típiques d'aquests kernels.

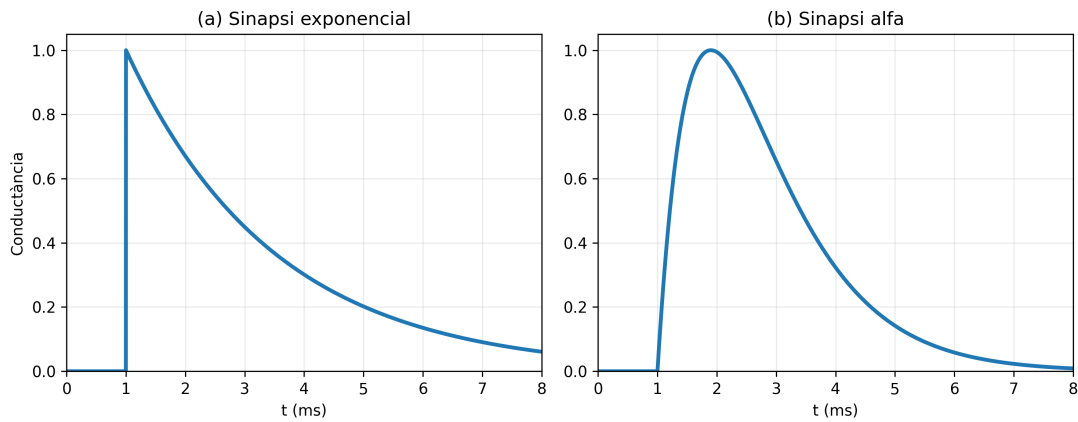


Figura 16: Resposta temporal de la conductància sinàptica per a (a) una sinapsi exponencial i (b) una sinapsi alfa. Font: [31]

Aquests tipus de kernel introdueixen una memòria temporal a la sinapsi, ja que l'efecte d'un pols persisteix durant un cert interval de temps abans de decaure. Això implica que el potencial de membrana d'una neurona reflecteix no només el nombre de polsos entrants, sinó també la seva distribució temporal.

En el context de neurones LIF, el corrent sinàptic s'integra conjuntament amb el mecanisme de fuita o leak de la membrana, donant lloc a una dinàmica que depèn de la interacció entre la constant de temps de la sinapsi τ_s i la constant de temps de la membrana τ_m . Aquesta doble escala temporal permet diferenciar patrons de polsos que arriben amb diferències temporals petites.

A més de la forma temporal del corrent, moltes formulacions inclouen explícitament un retard sinàptic associat a cada connexió. Aquest retard simula el temps necessari perquè un spike es propagui des de la neurona presinàptica fins a la postsinàptica, i es pot incloure modificant l'expressió del corrent sinàptic com:

$$I_{syn}(t) = \sum_{i,j} w_{ij} \sum_k \kappa(t - t_j^k - d_{ij}) \quad (9)$$

on d_{ij} és el retard sinàptic de la connexió $j \rightarrow i$. La introducció de retards permet que la xarxa sigui sensible no només a la presència de polsos, sinó també a la seva posició temporal, fent possible implementar mecanismes de detecció de patrons temporals, sincronització i càlcul basat en l'ordre d'arribada dels senyals.

En una formulació discreta en el temps, com per exemple en simulacions, el corrent sinàptic pot representar-se mitjançant una variable d'estat que es va actualitzant:

$$I_{syn}(t+1) = \beta I_{syn}(t) + \sum_j w_{ij} s_j(t - d_{ij}) \quad (10)$$

on $\beta = e^{-\Delta t/\tau_s}$ i $s_j(t) \in \{0, 1\}$ indica la presència d'un pols en el pas temporal. Aquesta expressió mostra com els polsos entrants contribueixen al corrent de manera acumulativa i amb una memòria temporal finita.

En conjunt, el model sinàptic constitueix l'enllaç entre la dinàmica individual de les neurones i el comportament col·lectiu de la xarxa. Mitjançant la combinació de pesos sinàptics, kernels i retards de propagació, les SNNs poden entendre i utilitzar informació distribuïda en el temps i explotar la dimensió temporal com a part del càlcul neuronal, diferenciant-se així de les xarxes neuronals artificials convencionals.

4.4 Codificació i entrenament en xarxes neuronals de polsos

En les SNNs, la informació no es representa principalment mitjançant valors continus, sinó a través de seqüències de polsos discrets en el temps, anomenats spikes. Això implica que tant les entrades com les sortides del sistema han de ser codificades temporalment, i que la dinàmica interna de la xarxa depèn de la interacció entre aquests polsos i la memòria temporal de les neurones. La manera com es realitza aquesta codificació és un element fonamental en el disseny d'una SNN i condiciona tant el comportament dinàmic com l'estratègia d'entrenament.

Un dels primers aspectes clau en les SNNs és la codificació de la informació. La codificació defineix com un senyal, ja sigui continu o discret, es transforma en un tren de polsos que pot ser processat per la xarxa. Una de les estratègies més simples i utilitzades és la codificació per taxa (rate coding), en la qual la informació es representa mitjançant la freqüència de dispar d'una neurona al llarg d'un tram de temps. En aquest esquema, un valor elevat es correspon amb una taxa de dispar alta, mentre que un valor baix genera pocs o cap pols. Aquest tipus de codificació té una interpretació intuïtiva i és robust davant el soroll, ja que petites variacions en el temps dels polsos no afecten significativament la taxa mitjana. No obstant això, el rate coding sol requerir trams relativament llargs per obtenir una estimació fiable de la taxa, fet que pot incrementar la latència i el cost computacional. En la pràctica, la codificació per taxa es sol implementar generant trens de polsos segons un procés estocàstic, habitualment de tipus Poisson, on la probabilitat d'emetre un pols en un interval de temps discret és proporcional al valor de l'entrada normalitzada. Aquest enfocament presenta l'avantatge de ser senzill d'implementar i relativament tolerant al soroll, però requereix finestres temporals prou llargues perquè la taxa d'actuació estimada sigui representativa del valor original. Malgrat aquesta limitació, el rate encoding resulta especialment adequat en escenaris de conversió de pesos ANN-SNN, ja que preserva la semàntica de les activacions mitjanes del model original.

Una altra estratègia de codificació són les codificacions temporals, que aprofiten de manera més directa el temps d'ocurrència dels polsos. En aquests esquemes, la informació no es codifica en el nombre total de polsos, sinó en els instants exactes en què aquests es produeixen o en les relacions temporals entre diferents neurones. Un exemple representatiu és la codificació *time-to-first-spike*, on el valor d'un estímul es reflecteix en el retard fins al primer pols després d'un instant de referència. Altres variants inclouen la codificació per latència, que consisteix a representar la informació mitjançant el temps relatiu d'actuació respecte a un esdeveniment o estímul concret, la codificació per fase respecte a una oscil·lació global, on els polsos es posicionen dins d'un cicle oscil·latori i la fase en què es produeix l'actuació conté la informació, o la codificació basada en l'ordre d'actuació de diferents neurones, en la qual el patró informatiu ve determinat per la seqüència relativa de disparells, independentment del temps absolut. Aquestes estratègies permeten una representació molt més eficient i ràpida de la informació, sovint amb un nombre reduït de polsos, però són més sensibles al soroll temporal i requereixen una sincronització més acurada dels senyals. Les codificacions temporals s'utilitzen principalment en escenaris on la latència i l'eficiència són factors crítics, o quan la naturalesa del senyal ja és inherentment temporal. Són especialment rellevants en el processament sensorial basat en esdeveniments, com ara visió neuromòrfica amb sensors dinàmics de visió (DVS), processament auditiu o sistemes que han de reaccionar en temps real amb un nombre mínim d'operacions. En aquests contextos, el fet de poder prendre decisions basades en els primers polsos permet reduir significativament el temps de resposta i el consum energètic. A més, aquestes codificacions exploten de manera més plena la dinàmica temporal de les neurones, apropant el funcionament del model a certs mecanismes observats en sistemes neuronals biològics. No obstant això, el seu ús acostuma a implicar un disseny més precís de la xarxa i dels mecanismes de sincronització, ja que petites variacions temporals poden afectar de manera significativa la representació de la informació. A la [Figura 17](#) es pot observar una comparació entre la codificació per taxa i diferents esquemes de codificació temporal, il·lustrant com cada mètode representa la mateixa informació mitjançant diferents patrons de polsos.

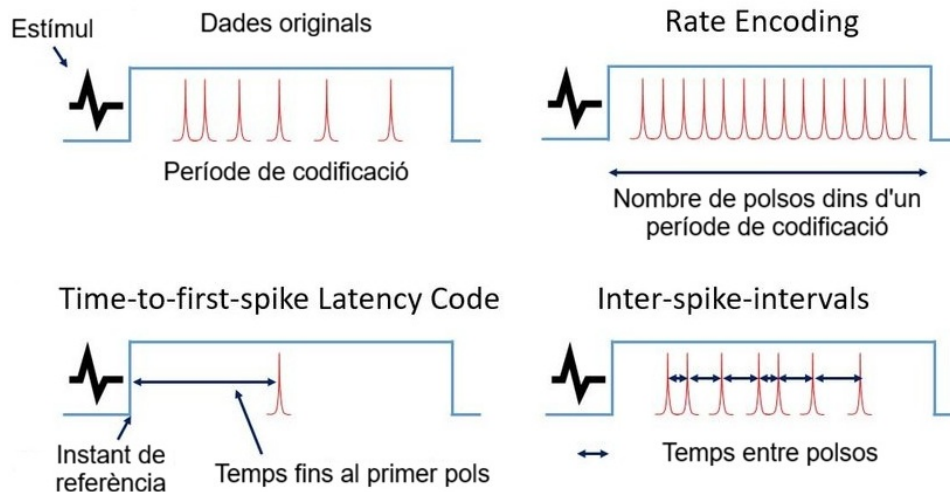


Figura 17: Comparació entre codificació per taxa i codificacions temporals. Font: [32]

Tot i que la codificació per taxa i les codificacions temporals són les més utilitzades avui dia, existeixen altres esquemes de codificació menys comuns però rellevants en determinats contextos. Un exemple és el *burst coding*, en el qual la informació es representa mitjançant ràfegues de polsos emesos en intervals de temps molt curts. En aquest cas, no només és important la presència del dispar, sinó també el nombre de polsos dins d'una ràfega i la seva separació temporal. Aquest tipus de codificació s'ha utilitzat per modelar fenòmens biològics associats a l'atenció o a la detecció d'estímuls rellevants, i pot aportar una major robustesa davant el soroll en entorns on els estímuls són transitoris.

Un altre esquema alternatiu és la *delta modulation*, on els polsos no codifiquen el valor absolut d'un senyal, sinó els canvis o increments respecte a un estat anterior. En aquest enfocament, un pols s'emet quan la variació del senyal supera un cert llindar, la qual cosa el fa especialment adequat per a senyals amb dinàmiques suaus o per a sistemes orientats a l'eficiència energètica. Aquest tipus de codificació és habitual en sistemes basats en esdeveniments, ja que permet reduir significativament el nombre de polsos generats quan el senyal és estable.

4.5 Entrenament de xarxes neuronals de polsos

L'aprenentatge de les SNNs presenta un repte addicional respecte al de les ANN convencionals, ja que la funció de dispar de la neurona és inherentment no derivable, i per tant no es pot emprar el mètode de la retropropagació. Una de les aproximacions més estudiades per abordar aquest problema és l'ús de regles d'aprenentatge locals inspirades en la neurobiologia, com la *Spike-Timing-Dependent Plasticity* (STDP). En l'STDP, els pesos sinàptics s'ajusten en funció de la diferència temporal entre els polsos presinàptics i postsinàptics. Si un pols presinàptic precedeix un pols postsinàptic dins d'una període de temps curt, el pes sinàptic tendeix a augmentar, reforçant una relació causal. En canvi, si l'ordre temporal és invers, el pes es redueix. Aquest tipus d'aprenentatge és completament local, ja que depèn només de la informació disponible a la sinapsi, i permet capturar correlacions temporals sense necessitat d'un senyal d'error global.

Tot i els seus avantatges en termes de plausibilitat biològica i eficiència, l'STDP presenta limitacions importants quan s'aplica a tasques supervisades complexes. Per aquest motiu, s'han desenvolupat extensions com l'STDP modulada per recompensa, en la qual les actualitzacions

sinàptiques locals es combinen amb un senyal global que reflecteix l'èxit o el fracàs del comportament de la xarxa. Aquest enfocament permet aplicar les SNNs a escenaris d'aprenentatge per reforç, on la recompensa pot estar retardada en el temps.

En paral·lel, una altra línia d'investigació molt activa es centra en l'adaptació dels mètodes de retropropagació del gradient a les SNNs mitjançant l'ús de gradients substitutius (surrogate gradients). En aquest enfocament, la funció de dispar, que és discontinua, es manté durant la simulació directa de la xarxa, però durant la fase de retropropagació se'n substitueix la derivada per una funció suau artificial. Aquesta aproximació permet aplicar una retropropagació a través del temps, o *backpropagation-through-time*, sobre la dinàmica temporal de la xarxa i entrenar arquitectures profundes de SNN de manera supervisada. Malgrat que aquest mètode sacrifica part de la plausibilitat biològica, ofereix una gran flexibilitat i ha demostrat ser molt eficaç en tasques de classificació, processament de sèries temporals i aprenentatge seqüencial.

L'elecció entre codificació per taxa o codificació temporal, així com entre aprenentatge local basat en STDP o entrenament amb gradients substitutius, depèn fortament del tipus d'aplicació i dels requisits del sistema. Les estratègies basades en taxa solen ser més estables i senzilles, mentre que les codificacions temporals permeten aprofitar millor el potencial de càlcul temporal de les SNNs. De la mateixa manera, els mètodes d'aprenentatge local són especialment adequats per a sistemes distribuïts i implementacions neuromòrfiques, mentre que els mètodes basats en gradients ofereixen una major capacitat d'optimització global en entorns supervisats.

En conjunt, aquests mecanismes de codificació, dinàmica sinàptica i aprenentatge constitueixen els fonaments tècnics de les xarxes neuronals espigades. La combinació adequada d'aquests elements permet adaptar les SNNs a una àmplia varietat d'aplicacions, des del processament sensorial basat en esdeveniments fins a la classificació i el control en temps real, tot aprofitant la dimensió temporal com a part intrínseca del càlcul neuronal.

Una altra estratègia rellevant per a l'entrenament de xarxes neuronals de polsos consisteix a entrenar prèviament una xarxa neuronal artificial convencional i, posteriorment, transferir els pesos i biaixos a una SNN. En aquest enfocament, la ANN s'entrena utilitzant mètodes estàndard de retropropagació i funcions d'activació contínues, com ara la funció ReLU, amb les dades d'entrada i sortida transformades per ser compatibles amb la SNN (entrades i sortides $\in [0, 1]$), mentre que la SNN hereda els paràmetres apresos. Aquesta conversió es fonamenta en l'equivalència funcional entre les activacions mitjanes d'una ANN i la taxa de dispar d'una neurona de polsos, de manera que els pesos sinàptics poden conservar, almenys de forma aproximada, el comportament del model original.

$$z = \sum_j w_j x_j + b \quad \Longleftrightarrow \quad V(t+1) = \alpha V(t) + \sum_j w_j s_j(t) + b \quad (11)$$

El procés de conversió requereix habitualment una normalització dels pesos i una definició acurada dels llindars de dispar de les neurones de polsos, amb l'objectiu d'evitar saturacions i garantir que les taxes d'actuació es mantinguin dins d'un rang operatiu adequat. Els biaixos de la ANN es poden interpretar com a corrents constants o desplaçaments del potencial de membrana en la SNN, integrant-se així en la dinàmica temporal del model. Un cop realitzada la transferència, la SNN pot operar sense necessitat d'entrenament addicional, o bé refinar-se mitjançant tècniques d'ajustament (fine-tuning) basades en mètodes específics de SNN com els ja

mentzionats.

Aquest enfocament presenta diversos avantatges. En primer lloc, permet aprofitar la estabilitat dels mètodes d'entrenament de les ANN, evitant les dificultats associades a la no diferenciabilitat del dispar de les SNNs. A més, facilita l'ús de SNNs en tasques on les ANN ja han demostrat un rendiment elevat, proporcionant una via directa per beneficiar-se del potencial estalvi energètic i del processament temporal propi de les xarxes de polsos. No obstant això, aquesta estratègia també presenta limitacions. La conversió sol requerir finestres temporals prou llargues perquè les taxes de dispar convergeixin cap a les activacions equivalents, la qual cosa pot incrementar la latència. A més, no totes les dinàmiques temporals pròpies de les SNNs són explotades en aquest enfocament, ja que el comportament final queda condicionat pel model d'ANN original. Per aquest motiu, l'entrenament per conversió ANN-SNN s'utilitza principalment com una estratègia intermitja entre els mètodes clàssics de deep learning i els enfocaments específics de SNN, oferint un compromís entre rendiment, simplicitat i eficiència computacional. Per posar a prova l'eficàcia d'aquest tipus de xarxa, i més concretament de la conversió ANN-SNN, al següent apartat es descriu el desenvolupament pràctic d'una ANN entrenada per la classificació d'un conjunt de dades, la seva posterior conversió a SNN i l'avaluació del rendiment obtingut.

5 Desenvolupament pràctic: Classificació amb ANN i SNN

Per tal de posar a prova els fonaments tècnics exposats en el capítol anterior i avaluar la viabilitat de les SNNs en escenaris reals, la part pràctica d'aquest treball es centra en l'estudi comparatiu entre xarxes neuronals artificials convencionals i xarxes neuronals de polsos aplicades a una tasca de classificació. L'objectiu és analitzar fins a quin punt les SNNs poden reproduir el comportament funcional d'un model ANN ja entrenada, així com identificar els avantatges i limitacions que presenta aquesta transició.

En primer lloc, s'ha entrenat una petita xarxa neuronal artificial per actuar com a classificador de dades, utilitzant mètodes d'entrenament estàndard basats en la retropropagació del gradient. Aquest model ANN serveix com a punt de referència. Un cop obtingut el model, els pesos i biaixos apresos s'han utilitzat com a base per a la construcció d'una xarxa neuronal de polsos equivalent.

A partir d'aquest model inicial, s'ha dut a terme la transformació dels paràmetres de la ANN cap a una SNN, reinterpretant els pesos sinàptics i els termes de biaix dins d'un marc temporal basat en polsos discrets. La SNN resultant s'ha avaluat sota les mateixes condicions que el model original, considerant criteris com la precisió i la exactitud en la classificació de les dades, així com la eficiència computacional.

Finalment, els resultats obtinguts amb ambdós models s'han comparat de manera sistemàtica, amb l'objectiu d'analitzar les diferències de rendiment i de funcionament entre els dos. Aquesta comparativa permet valorar en quines condicions les SNNs poden esdevenir una alternativa viable a les ANN convencionals, especialment en contextos on el processament temporal o l'eficiència computacional són factors rellevants. D'aquesta manera, la part pràctica no només valida els conceptes teòrics presentats, sinó que també ofereix una visió aplicada del potencial de les SNNs en la indústria moderna.

5.1 Generació i transformació de les dades d'entrenament

Per poder portar a terme l'entrenament de l'ANN, primer hem de generar i adaptar les dades a utilitzar. Per fer-ho, s'ha utilitzat el programa de MATLAB, doncs degut a les seves eines gràfiques integrades que simplifiquen la manipulació geomètrica, i a la seva funció nativa *inpolygon*, que permet, amb una sola crida, determinar si un punt cau dins d'un polígon arbitrari, evitant així la necessitat d'implementar algoritmes de detecció de punts en polígon des de zero, el fan l'entorn ideal per a aquesta tasca. A més, MATLAB facilita la generació aleatòria de vectors i matrius i la interpolació de vèrtexs, funcions essencials per la generació de les dades necessàries.

Així doncs, s'ha creat un script que agafa com a entrada la quantitat de vèrtex que ha de tenir el polígon, i la quantitat de punts del perímetre a retornar, i retorna aquesta quantitat p de punts en una matriu $2 \times p$ d'un polígon pseudo-aleatòriament generat. Aquest no és realment aleatori degut a les limitacions inherents dels ordinadors, que són màquines deterministes, i per tant qualsevol generador basat només en programari produeix pseudo-aleatorietat i mai serà completament aleatori. Per a l'objectiu de generar polígons però, aquesta pseudo-aleatorietat és suficient. Un cop creat l'escript, s'executa per generar el següent polígon, que es mostra a la [Figura 18](#).

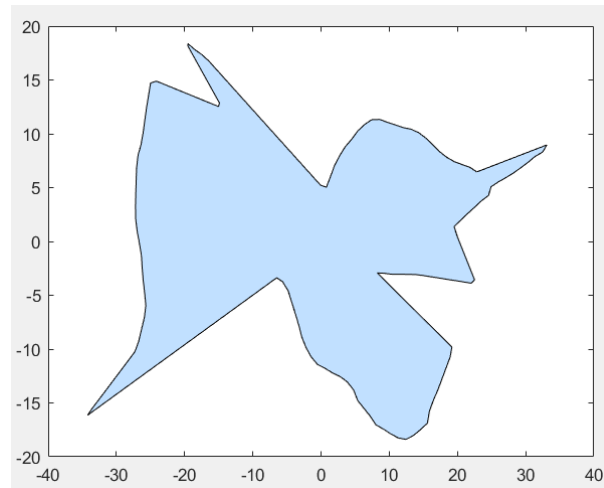


Figura 18: Polígon generat per l'script de MATLAB.

Una vegada generat el polígon que s'utilitzarà com a frontera de classificació, s'ha generat un altre script per generar les dades que s'utilitzaran per l'entrenament de l'ANN. Aquest script agafa com a entrada la quantitat de punts a generar i el polígon, expressat en punts del seu perímetre, i retorna una matriu $3 \times p$, on cada columna representa un punt de coordenades (x,y) i la seva classe associada (0 o 1, depenent de si el punt cau dins o fora del polígon). La generació dels punts es realitza de manera pseudo-aleatòria, fent-se servir la funció nativa de MATLAB *rand* per generar punts uniformement distribuïts dins d'un quadrat que engloba el polígon, i de la funció *inpolygon* per determinar si cada punt cau dins o fora del polígon. Un cop obtinguda la matriu de dades, l'escript les guarda en un fitxer .csv que posteriorment serà carregat per l'entrenament de l'ANN, i genera una figura que mostra els punts generats, amb diferents colors segons la classe a la que pertanyen, com es pot veure a la [Figura 19](#).

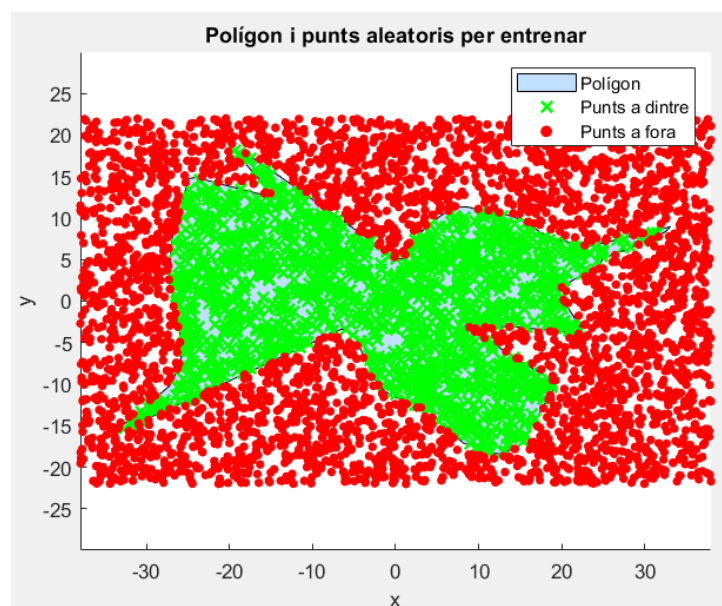


Figura 19: Dades d'entrenament generades, amb punts de classe 0 (fora del polígon) en vermell i punts de classe 1 (dins del polígon) en vert.

Amb el conjunt de dades exportat en format .csv, aquest es carrega a Python mitjançant el fitxer anomenat Dades.py, encarregat de dur a terme tot el preprocessament previ a l'entrenament de la xarxa neuronal. En primer lloc, aquest fitxer carrega les dades del .csv fent-se ajuda de la llibreria pandas, extreu les coordenades i les classes dels punts, i les ordena en columnes separades, de nom x i y per les coordenades, i Sortida per les classes. Tot seguit, les dades es converteixen al format numèric float32, format ampliament utilitzat en el deep learning degut a la seva eficiència computacional, i es normalitzen linealment les coordenades al rang $[0,1]$, fent correspondre l'interval $x \in [-40, 40]$ i l'interval $y \in [-20, 20]$ a aquest mateix rang, intervals que es corresponen amb els utilitzats en la generació de les dades. En l'entrenament d'una ANN, i més concretament d'un MLP, com és el cas, no és necessari fer aquesta normalització, doncs els pesos i biaixos de la xarxa poden adaptar-se a les escales de les dades d'entrada. No obstant això, aquesta normalització es realitza per possibilitar la posterior conversió a SNN, ja que aquests tipus de xarxes requereixen que les entrades i sortides es trobin dins d'un rang específic, com ara $[0, 1]$, ja que els polsos que generen les neurones de polsos es poden interpretar com a valors binaris, i la taxa de dispar d'una neurona de polsos es correspon amb el valor de l'activació mitjana d'una ANN, per tant, per mantenir aquesta equivalència funcional, és necessari que les dades d'entrada i sortida es trobin dins d'aquest rang. Un cop normalitzades, tant les dades d'entrada com les etiquetes es transformen en tensors de PyTorch, format que permet utilitzar les eines d'aquesta llibreria per a l'entrenament de la xarxa neuronal, i es divideixen de manera aleatòria en subconjunts d'entrenament, validació i test. Aquest pas evita que les mostres mantinguin l'ordre original del fitxer i garanteix una distribució més representativa dels exemples en cada subconjunt, alhora que permet avaluar de manera més fiable el rendiment del model en dades no vistes durant l'entrenament. Finalment, el fitxer Dades.py retorna els tensors corresponents als subconjunts d'entrenament, validació i test, que seran utilitzats en la fase d'entrenament de l'ANN.

5.2 Creació, entrenament i avaluació de la ANN

Un cop processades les dades, es procedeix a la definició i entrenament d'una xarxa neuronal artificial convencional, més concretament un MLP, que actuï com a classificador binari dels punts generats. La implementació s'ha dut a terme en Python utilitzant la llibreria PyTorch, escollida per la seva flexibilitat i facilitat d'ús en el desenvolupament de models de deep learning, així com per la seva àmplia comunitat i suport. Es comença definint una classe que hereta de *torch.nn.Module*, que representa el model de xarxa neuronal. Aquesta classe inclou la definició de les capes lineals i les funcions d'activació, així com el mètode *forward* que especifica com es processa la informació a través de la xarxa. Per aquest cas i tenint en compte que la finalitat del primer model és demostrar la viabilitat de la conversió a SNN, s'ha optat per una arquitectura senzilla, composta per una capa d'entrada de 2 neurones (corresponent a les coordenades x i y), una capa oculta de 6 neurones amb funció d'activació ReLU, i una capa de sortida de 1 neurona també amb funció d'activació ReLU. Per la tasca a realitzar per la ANN, una alternativa potser més adient hagués estat triar la funció d'activació sigmoide a la capa de sortida, ja que aquesta funció comprimeix les sortides al rang $[0,1]$, facilitant la interpretació de la sortida. No obstant això, s'ha optat per la funció ReLU per mantenir una coherència amb la posterior conversió a SNN, doncs les sortides de la ANN es poden interpretar com a taxes de dispar d'una SNN, i la modificació que aplica la funció d'activació Sigmoide sobre aquesta és no lineal, cosa que afectaria negativament al procés de conversió, mentre que la funció ReLU manté una relació més directa amb la conversió a SNN, només eliminant els nombres negatius, i deixant les sortides positives intactes. Un cop definida l'arquitectura del model, es procedeix a l'entrenament utilitzant un optimitzador de tipus Adam i una funció de pèrdua de tipus Mean Squared Error

(MSE). Per una tasca de classificació binària, tornem a trobar que existeix una funció més adient, com és la Binary Cross-Entropy (BCE), però de nou obtem per una que facilita el procés de conversió, doncs a diferència d'un model entrenat amb la funció BCE, un entrenat amb MSE no optimitza per una probabilitat logística "pura", sinó per un valor escalar de resposta (en aquest cas 0 o 1), cosa que assimila més el seu comportament al d'un model SNN. El procés d'entrenament es realitza durant un nombre determinat d'èpoques, durant les quals el model ajusta els seus pesos i biaixos per minimitzar la pèrdua en el conjunt d'entrenament. A aquesta quantitat arbitrària d'èpoques se li aplica una funció d'aturada prematura, que mesura el rendiment del model en el subconjunt de validació després de cada època, el mostra, i si detecta que aquesta no millora, l'atura. Durant el procés d'entrenament, es registren en un arxiu separat anomenat `entrenament_log` les mètriques de rendiment, com ara la pèrdua i l'exactitud, tant en el conjunt d'entrenament com en el de validació, per a cada època. Es pot veure un exemple de les mètriques registrades en l'arxiu `entrenament_log` al [Listing 1](#).

```
Comencant entrenament de la xarxa.
Neurones d'entrada: 2
Neurones de capes ocultes: 6
Capes ocultes: 1
Neurones de sortida: 1
Taxa d'aprenentatge: 0.001
Decaiment de pesos: 1e-05
Funcio de perdua: Mean Squared Error Loss
Optimitzador: Adam
Epoques: 200

Epoca 01   train loss: 0.2143   val loss: 0.1987
Epoca 02   train loss: 0.1761   val loss: 0.1542
Epoca 03   train loss: 0.1328   val loss: 0.1194
...
Test Loss: 0.0412, Percentatge d'encert: 94.37 %
```

Listing 1: Fragment dels logs d'entrenament de la ANN

Un cop finalitzat l'entrenament, el model es guarda en un fitxer anomenat `Pesos_classificador.pth`, que conté els pesos i biaixos apresos, i es procedeix a avaluar el rendiment del model en el subconjunt de test, obtenint una mètrica d'exactitud que serveix com a referència per a la posterior comparació amb la SNN convertida.

Per tal de comprovar de manera més intuïtiva i visual el comportament del model entrenat, s'ha desenvolupat un segon arxiu en Python anomenat `Test_model.py`, destinat específicament a la fase de prova de la xarxa neuronal. Aquest script carrega el model prèviament entrenat i guardat al fitxer de pesos i restaura l'estat dels paràmetres apresos durant l'entrenament. Un cop carregat, el model es posa en mode d'avaluació, cosa que garanteix que la xarxa treballi exclusivament en fase d'inferència i no modifiqui els seus pesos interns. A partir d'aquí, l'script permet a l'usuari introduir manualment les coordenades d'un punt concret del pla, i retrona la predicció que realitza l'ANN. Per fer-ho, les coordenades introduïdes es processen de la mateixa manera que durant la fase d'entrenament. Primer es normalitzen al rang $[0,1]$, mantenint la coherència amb el preprocessament aplicat al conjunt de dades original, i posteriorment es transformen en un tensor de PyTorch amb el format adequat perquè pugui ser processat pel model. A més, l'script incorpora el càlcul del valor màxim de sortida obtingut sobre el conjunt de validació, que s'utilitza com a factor de normalització per expressar la resposta del model

d'una manera més interpretable, es a dir, en un rang de $[0,1]$, igual que amb les dades d'entrada. D'aquesta manera, en lloc de mostrar únicament la sortida bruta de la xarxa, es presenta una magnitud normalitzada que es pot interpretar com una probabilitat aproximada que el punt es trobi dins del polígon. Aquesta normalització permet entendre millor la resposta del model, doncs es veu com, al apropar-se al límit del polígon, la sortida va disminuint progressivament, i com els punts que es troben clarament dins del polígon obtenen una resposta propera a 1, mentre que els punts clarament fora del polígon obtenen una resposta propera a 0, sent els punts de frontera els que obtenen respostes intermitges.

5.3 Conversió a SNN

Un cop entrenada i validada la xarxa neuronal artificial convencional, el següent pas consisteix en la seva conversió a una xarxa neuronal de polsos. Aquesta conversió s'ha implementat al fitxer `SNN_model.py`, on es carrega el model ANN entrenat, se n'extreuen els pesos i biaixos, i es redefineix el càlcul de la xarxa dins d'un marc temporal discret basat en polsos. L'objectiu d'aquesta etapa no és tornar a entrenar el model des de zero, sinó conservar tant com sigui possible el comportament funcional de la ANN original i reinterpretar-lo. Per fer-ho, es parteix directament del classificador definit a `Train_model.py`, amb arquitectura $2 \rightarrow 6 \rightarrow 1$, i se'n carreguen els pesos guardats després de l'entrenament. La conversió es basa en l'equivalència de funcionament entre l'activació d'una neurona en una ANN i la taxa de dispar d'una neurona en una SNN. En lloc de treballar amb sortides contínues calculades en un únic pas, la SNN replica la mateixa transformació mitjançant una seqüència de passos temporals, en què les neurones acumulen corrent d'entrada, apliquen una dinàmica de memòria amb fuga i generen un pols quan el potencial de membrana supera un llindar determinat, com s'explica amb més detall a l'apartat anterior. El primer element important de la conversió és la codificació de les entrades. En el model implementat no s'utilitza una codificació temporal complexa, sinó una codificació per taxa o *rate encoding*. A cada pas temporal, cada coordenada d'entrada, ja normalitzada al rang $[0,1]$, es transforma en un pols binari segons una regla estocàstica simple es genera un nombre aleatori uniforme i es compara amb el valor de l'entrada, de manera que com més gran és aquest valor, més probable és que la neurona d'entrada dispari en aquell instant. Això implica que una entrada propera a 1 generarà molts més polsos al llarg d'un interval de temps determinat que una entrada propera a 0. En conseqüència, la informació no es codifica en la magnitud directa del senyal, sinó en la freqüència relativa de polsos produïda durant els passos de simulació. En el codi, aquesta operació es realitza mitjançant l'expressió `(torch.rand_like(x_input) < x_input).float()`, que constitueix el mecanisme de codificació d'entrada de la SNN. Al [Listing 2](#) es pot veure un fragment del codi on es realitza aquesta codificació per taxa de les entrades, i com aquest modifica els paràmetres interns de la xarxa SNN.

```

def forward(self, x_input):

    # --- CAPA 1 (Entrada -> Oculta 1) ---

    threshold= 1.0      # Llindar sortida
    decay = 0.9         # Leak (mantenim memoria)

    v1 = torch.zeros(self.w1.size(0)) # Inicialitzem potencials de membrana a
zero
    v2 = torch.zeros(self.w2.size(0)) # Inicialitzem potencials de membrana a
zero

    dispars_totals = 0

    for pas in range(self.t):

        # Input rate coding
        input_spikes = (torch.rand_like(x_input) < x_input).float()

        # CAPA OCULTA
        corrent1 = torch.matmul(input_spikes, self.w1.T)+ self.b1
        v1 = v1 * decay + corrent1

        spikes1 = (v1 >= threshold).float()
        v1 = v1 - (spikes1 * threshold) # Reset amb el threshold

        # CAPA SORTIDA

        corrent2 = torch.matmul(spikes1, self.w2.T) + self.b2
        v2 = v2 * decay + corrent2

        spike_out = (v2 >= threshold).float()
        v2 = v2 - (spike_out * threshold)

        if spike_out > 0:
            dispars_totals += 1

```

Listing 2: Fragment de la codificació per taxa de les entrades, paràmetres interns i sortides

El segon element important que apareix és la normalització dels pesos i dels biaixos de la ANN abans d'introduir-los a la SNN. En el codi, abans de crear la xarxa de polsos, es calcula l'activació màxima observada a la capa oculta i a la capa de sortida del model original sobre el conjunt de validació. Aquests valors, anomenats `a1_max` i `a2_max`, s'utilitzen com a factors de normalització per dividir, respectivament, els pesos i els biaixos de la primera capa i els de la capa de sortida. Això permet reduir l'amplitud efectiva dels corrents injectats a les neurones de polsos i evitar que el potencial de membrana saturi massa ràpidament. En altres paraules, aquesta normalització serveix per adaptar l'escala d'activació de la ANN a l'escala de la SNN, on les neurones només poden emetre polsos quan superen un llindar fix. Si no es dugués a terme aquesta etapa, seria probable que algunes neurones desapareixin de manera gairebé contínua o, a l'inrevés, que no arribessin mai a superar el llindar, degradant significativament el comportament del model convertit. Un cop codificades les entrades i normalitzats els paràmetres, la dinàmica interna de la SNN es construeix amb dues capes de neurones de polsos. La

primera rep els polsos d'entrada i calcula un corrent sinàptic a partir del producte entre els polsos i la matriu de pesos W_1 , sumant-hi el vector de biaix b_1 . El potencial de membrana de cada neurona es va actualitzant en el temps segons una dinàmica amb fuga, modelada per un factor $\text{decay} = 0.9$. Quan aquest potencial supera el llindar fixat a $\text{threshold} = 1.0$, la neurona emet un pols i el potencial es redueix restant aquest llindar ($0.0 + \text{biaix}$), implementant així un mecanisme de reset. La capa de sortida funciona de la mateixa manera, rebent com a entrada els polsos de la capa oculta i acumulant-los temporalment fins que genera els polsos finals de sortida. La resposta del model no és una probabilitat contínua com en el cas de la ANN, sinó la proporció de polsos emesos per la neurona de sortida al llarg de l'interval de temps, és a dir, $\text{dispars_totals} / t$. En l'avaluació final, la implementació utilitza una finestra de 200 passos temporals. Al [Listing 3](#) es pot veure un fragment del codi on es realitza aquesta normalització dels pesos i biaixos a les diferents capes de la SNN.

```
class SNN():
    def __init__(self, mlp_model, a1_max, a2_max, passos_temps=50):
        self.t = passos_temps

        # Extraccio de pesos i biaixos de l'MLP per a l'SNN:

        # Capa 1
        self.w1 = mlp_model.net[0].weight.data / a1_max # Matriu 6x2
        self.b1 = mlp_model.net[0].bias.data / a1_max   # Vector 6

        # Capa Sortida
        self.w2 = mlp_model.net[2].weight.data / a2_max # Matriu 1x6
        self.b2 = mlp_model.net[2].bias.data / a2_max   # Vector 1
```

Listing 3: Fragment de la implementació de la classe SNN utilitzada i conversió de pesos i biaixos de l'ANN a l'SNN

Per avaluar la SNN convertida, el codi inclou una funció específica d'*accuracy*, que recorre el conjunt de prova i, per a cada punt, calcula la sortida de la SNN, la compara amb un llindar de decisió i n'obté la classe predita. A diferència de la ANN, que en l'avaluació base utilitza un llindar de 0.5, per a la SNN un llindar de 0.59 per compensar la diferència en l'escala de sortida, doncs al haver utilitzat la funció ReLU a la ANN, l'output d'aquesta podia donar valors superiors a 1, i per tant el threshold de 0,5 no és exactament la meitat, i hem de compensar a la SNN per assimilar la seva precisió. Aquesta mena de conversió acostuma a preservar raonablement bé el comportament de la xarxa en el nucli principal de cada regió de classificació, és a dir, en aquelles zones del pla on la ANN original produeix activacions clarament per sobre o clarament per sota del llindar. En aquestes regions, la separació entre classes és prou robusta perquè les fluctuacions introduïdes pel *rate encoding* i per la discretització temporal tinguin un efecte limitat. Per contra, és esperable que les pèrdues de precisió apareguin sobretot a prop de la frontera de decisió, on petits canvis en la taxa de dispar, en el nombre de passos temporals simulats o en el llindar final poden alterar la classe assignada al punt. Aquesta degradació és coherent amb el fet que la SNN implementada utilitza una codificació estocàstica i una finestra temporal finita. La sortida d'aquesta és una aproximació mitjana al comportament de la ANN, però no una reproducció exacta punt a punt, especialment en les zones més ambigües del problema. No obstant això, en la pràctica, aquesta aproximació pot ser suficient per a moltes aplicacions, especialment aquelles on la robustesa i l'eficiència energètica de les SNNs compensen les petites pèrdues de precisió en les zones de frontera. Cal remarcar que, per utilitats on la precisió

és crítica, és habitual aplicar després de la conversió un procés de reentrenament o ajustament (fine-tuning) de la SNN, que permet refinar els pesos i biaixos dins del marc temporal de la SNN per recuperar part de la precisió perduda durant la conversió inicial. En aquest cas, però, s'ha optat per no realitzar aquest pas addicional, per tal de mantenir el focus en la comparativa directa entre el model ANN original i la SNN convertida sense entrenament addicional.

5.4 Resultats i comparativa de rendiment

Amb els dos models entrenats, s'avaluen ambdós models sobre el conjunt de dades de validació amb l'objectiu d'obtenir una mesura del rendiment el menys esbiaixada possible. S'observa que la ANN obté un percentatge d'encert del 93.0%, mentre que la SNN convertida assoleix un 90.1%. Cal remarcar però que la precisió de la SNN varia en funció del període de temps en que s'evalui, sent més precisa com més temps se li doni per evaluar l'entrada. Per al nostre cas, i com un ordinador no entén de temps, sinó de cicles de càlcul, hem fet servir 200 cicles. A més, la precisió de la SNN també depèn d'un factor pseudo-aleatori, doncs la entrada que li donem no és una entrada real, sinó una transformació d'un punt a un encoding de polsos realitzats amb una funció aleatòria, el que implica que la sortida d'aquest model pot variar lleugerament entre execucions, afectant a la seva precisió. Tot i això la conversió ANN-SNN introdueix una pèrdua de rendiment relativament reduïda, d'aproximadament 2.9 punts percentuals. Aquesta diferència és coherent amb la pèrdua de precisió esperada al procés de conversió. Mentre que la ANN treballa directament amb valors continus, la SNN ha de representar aquesta informació mitjançant una codificació per taxa, una dinàmica temporal discreta i un recompte finit de polsos, fet que introdueix una certa aproximació respecte al comportament original del model. Per aquest motiu, és raonable que la xarxa convertida presenti una lleugera pèrdua de precisió respecte a la xarxa artificial entrenada directament. Tot i així, des d'un punt de vista qualitatiu, la diferència obtinguda és prou petita per considerar que els dos models són raonablement similars en termes de precisió. Això indica que la conversió conserva de manera satisfactòria el comportament funcional de la ANN i que la SNN resultant continua essent capaç de classificar correctament la gran majoria dels punts. Aquest resultat també apunta a una idea important. Si l'ANN de partida s'escalés a una arquitectura més gran amb l'objectiu d'aproximar encara més la seva precisió al 100%, és esperable que la SNN convertida també millorés en la mateixa direcció. D'aquesta manera, seria possible obtenir una SNN amb una precisió molt elevada i, alhora, beneficiar-se de les avantatges pròpies d'aquest tipus de xarxes, com ara la seva naturalesa temporal i el seu potencial d'eficiència computacional, sense necessitat d'entrenar-la directament com una xarxa de polsos.

6 Desenvolupament pràctic: Aplicació del dataset MNIST amb ANN i SNN com a simulació de cas d'ús real

6.1 Justificació de l'ús de MNIST

6.2 Preparació de dades i entrenament de la ANN

6.3 Conversió a SNN i configuració temporal

6.4 Resultats i comparativa entre ANN i SNN

7 Resultats

- Comparativa de rendiment: ANN vs SNN.
- Eficiència energètica estimada.
- Dificultats trobades durant el desenvolupament.
- Reflexió sobre la viabilitat de l'ús de SNNs a la pràctica.
- Possibles futurs usos.

8 Pressupost

9 Impacte ambiental

10 Conclusions

Agraïments

Gracies

Bibliografia

- [1] A. N. W. Bertrand Russell, *Principia Mathematica*, Vols. I-III. Cambridge University Press, 1910-1913.
- [2] W. McCulloch i W. Pitts, «A Logical Calculus of the Ideas Immanent in Nervous Activity,» *The Bulletin of Mathematical Biophysics*, 1943, Disponible a: <https://doi.org/10.1007/BF02478259>, Consultat: 20/08/2025.
- [3] A. Turing, «Computing Machinery and Intelligence,» *Mind*, vol. LIX, núm. 236, pàg. 433-460, 1950, Disponible a: <https://academic.oup.com/mind/article-abstract/LIX/236/433/986238>, Consultat: 20/08/2025.
- [4] J. McCarthy, M. L. Minsky, N. Rochester i C. E. Shannon, «A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence,» Dartmouth College, inf. tèc., 1956, Disponible a: <http://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html>, Consultat: 22/08/2025.
- [5] A. Newell, J. C. Shaw i H. A. Simon, «The Logic Theory Machine: A Complex Information Processing System,» a *Proceedings of the Western Joint Computer Conference*, Spartan Books, 1956, pàg. 99-103.
- [6] A. Newell, J. C. Shaw i H. A. Simon, «Report on a General Problem Solver,» Carnegie Institute of Technology, Technical Report Research Report, 1959, Disponible a: https://bitsavers.trailing-edge.com/pdf/rand/ipl/P-1584_Report_On_A_General_Problem-Solving_Program_Feb59.pdf, Consultat: 24/08/2025.
- [7] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers,» *IBM Journal of Research and Development*, vol. 3, núm. 3, pàg. 210-229, 1959, Disponible a: <https://ieeexplore.ieee.org/abstract/document/5392560>, Consultat: 27/08/2025.
- [8] F. Rosenblatt, «The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,» *Psychological Review*, vol. 65, núm. 6, pàg. 386-408, 1958, Disponible a: <https://psycnet.apa.org/record/1959-09865-001>, Consultat: 27/08/2025.
- [9] Georgetown University i IBM, *Demonstration of Automatic Translation - Georgetown-IBM Experiment*, Disponible a: https://www.ibm.com/ibm/history/exhibits/translation/translation_1.html, Consultat: 27/08/2025, 1954.
- [10] Bobrow i Daniel G., «STUDENT - A Computer Program for Solving Problems in Elementary Algebra,» Massachusetts Institute of Technology i Project MAC, Technical Report MIT Project MAC TR-41, 1964, Disponible a: <https://dspace.mit.edu/handle/1721.1/149326>, Consultat: 27/08/2025.
- [11] M. Minsky i S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.
- [12] J. T. Alan McCabe, «Handwritten Signature Verification Using Complementary Statistical Models,» *Journal of Computers*, Griffith University, 2009, Disponible a: https://www.researchgate.net/publication/42803645_Handwritten_Signature_Verification_Using_Complementary_Statistical_Models, Consultat: 02/10/2025.
- [13] D. E. Rumelhart, G. E. Hinton i R. J. Williams, «Learning representations by back-propagating errors,» *Nature*, vol. 323, núm. 6088, pàg. 533-536, 1986, Disponible a: <https://www.nature.com/articles/323533a0>, Consultat: 08/09/2025.

- [14] R. Hahnloser, H. Sprekeler i H. S. Seung, «Digital communications and the sigmoid function,» *Journal of Computational Neuroscience*, vol. 9, núm. 2, pàg. 195 - 203, 2000.
- [15] X. Glorot i Y. Bengio, «Deep Sparse Rectifier Neural Networks,» a *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, Disponible a: <https://aistats.org/aistats2011/proceedings.html>, Consultat: 14/09/2025, 2011, pàg. 315 - 323.
- [16] K. Fukushima, «Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position,» *Biological Cybernetics*, vol. 36, núm. 4, pàg. 193 - 202, 1980, Disponible a: <https://link.springer.com/article/10.1007/BF00344251>, Consultat: 14/09/2025.
- [17] K. Fukushima, «Visual Feature Extraction by a Multilayered Network of Analog Threshold Elements,» *IEEE Transactions on Systems Science and Cybernetics*, vol. 5, núm. 4, pàg. 322 - 333, 1969, Disponible a: <https://ieeexplore.ieee.org/document/4082265>, Consultat: 14/09/2025.
- [18] Y. LeCun, L. Bottou, Y. Bengio i P. Haffner, «Gradient-Based Learning Applied to Document Recognition,» *Proceedings of the IEEE*, vol. 86, núm. 11, pàg. 2278 - 2324, 1998, Disponible a: <https://ieeexplore.ieee.org/document/726791>, Consultat: 14/09/2025.
- [19] G. Jocher i et al., «YOLOv8: Open-Source Neural Networks for Object Detection,» *Zenodo*, 2023. DOI: [10.5281/zenodo.1234567](https://doi.org/10.5281/zenodo.1234567)
- [20] I. Goodfellow et al., «Generative Adversarial Nets,» a *Advances in Neural Information Processing Systems*, 2014.
- [21] A. Vaswani et al., «Attention is All you Need,» a *Advances in Neural Information Processing Systems*, I. Guyon et al., ed., vol. 30, Curran Associates, Inc., 2017.
- [22] T. Fawcett, «An introduction to ROC analysis,» *Pattern Recognition Letters*, vol. 27, núm. 8, pàg. 861 - 874, 2006.
- [23] K. Papineni, S. Roukos, T. Ward i W.-J. Zhu, «BLEU: a Method for Automatic Evaluation of Machine Translation,» a *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pàg. 311 - 318.
- [24] M. Everingham, L. Van Gool, C. K. Williams, J. Winn i A. Zisserman, «The PASCAL Visual Object Classes (VOC) Challenge,» *International Journal of Computer Vision*, vol. 88, núm. 2, pàg. 303 - 338, 2010.
- [25] S. Chen, «How much energy will AI really consume? The good, the bad and the unknown,» *Nature*, 2025, Disponible a: <https://www.nature.com/articles/d41586-025-00616-z>, Consultat: 10/10/2025.
- [26] A. L. Hodgkin i A. F. Huxley, «A quantitative description of membrane current and its application to conduction and excitation in nerve,» *The Journal of Physiology*, vol. 117, núm. 4, pàg. 500 - 544, 1952, Disponible a: https://www.nature.com/articles/nn1100_1165, Consultat: 15/12/2025.
- [27] «The equivalent circuit for the Hodgkin-Huxley equation,» *Upenn Laboratory of Biophysics Technical Report*, Disponible a: <https://www.sas.upenn.edu/LabManuals/BBB251/NIA/NEUROLAB/APPENDIX/EQUAT.HH/equivcrt.htm>, Consultat: 15/12/2025.
- [28] L. Lapicque, «Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation,» *Journal de Physiologie et de Pathologie Générale*, vol. 9, pàg. 620 - 635,

- 1907, Disponible a: <https://gallica.bnf.fr/ark:/12148/bpt6k96593z>, Consultat: 22/12/2025.
- [29] R. B. Stein, «A theoretical analysis of neuronal variability,» *Biophysical Journal*, vol. 5, núm. 2, pàg. 173 - 194, 1965, Disponible a: <https://www.sciencedirect.com/science/article/pii/S0006349565867449>, Consultat: 15/12/2025.
- [30] C. Lee, S. Sarwar, P. Panda, G. Srinivasan i K. Roy, «Enabling Spike-Based Backpropagation for Training Deep Neural Network Architectures,» *Frontiers in Neuroscience*, vol. 14, pàg. 119, febr. de 2020, Disponible a: https://www.researchgate.net/publication/339574089_Enabling_Spike-Based_Backpropagation_for_Training_Deep_Neural_Network_Architectures, Consultat: 22/12/2025.
- [31] D. Sterratt, B. Graham, A. Gillies i D. Willshaw, *Principles of Computational Modelling in Neuroscience*. Cambridge University Press, 2011, Disponible a: <https://www.compneuroprinciples.org/>, Consultat: 22/12/2025.
- [32] C. Zhao, J. Li, H. An i Y. Yi, «Energy efficient analog spiking temporal encoder with verification and recovery scheme for neuromorphic computing systems,» Disponible a: https://www.researchgate.net/figure/a-One-encoding-window-example-b-Rate-encoding-c-Time-to-first-spike-latency-code_fig1_316721913, Consultat: 20/12/2025, març de 2017, pàg. 138 - 143.

Anex

Codi

Logs d'entrenament

Esquemes Hardware